

CHAPTER-1

INTRODUCTION

1.Introduction

The Airline Reservation System is a web-based application designed to simplify and streamline the process of booking airline tickets. The system allows users to search for available flights, compare fares, make reservations, and manage bookings efficiently. It serves both customers and airline administrators, providing a user-friendly interface for travellers and a robust backend for managing flight schedules, seat availability, and pricing.

The main objective of this project is to automate the traditional flight booking process, reduce manual errors, and enhance the customer experience. Key features include user registration and login, real-time flight search and booking, e-ticket generation, cancellation, and payment gateway integration. The system also includes an admin module for managing flights, routes, user data, and reports.

Developed using modern technologies such as MySQL, Express.js, React.js, and Node.js (MERN stack), this project demonstrates the implementation of CRUD operations, secure authentication, and dynamic database connectivity. The Flight Reservation System ensures data accuracy, reliability, and scalability, making it a practical solution for airlines or travel agencies looking to digitize their operations.

a. Introduction of the System

Project Title:

The project title for this system is “AERO CONNECT”. This title captures the essence of the application connecting traveller’s to flights quickly and efficiently. The platform serves as a digital gateway for booking and managing flight reservations, emphasizing accessibility, real-time availability, and smooth user experience. The name symbolizes seamless air travel coordination, aiming to enhance how individuals and travel agencies book and manage air journeys.

Category:

“**AERO CONNECT**” falls under the category of travel technology and online reservation systems. This system utilizes modern web technologies to automate and simplify the booking process for air travel. Falling within the broader field of transportation and tourism management, this application contributes to digital transformation in the airline industry by promoting ease of access, efficient scheduling, and user-focused travel planning.

Overview:

“**AERO CONNECT**” is a full-featured Flight Reservation System Application developed to facilitate smooth, real-time flight bookings and travel management. Designed for both passengers and administrators, the system offers a user-friendly platform where individuals can search for flights, reserve tickets, choose seats, and receive booking confirmations—all within minutes. For airline or system administrators, the app includes robust tools for managing flight schedules, seat availability, passenger details, and pricing.

The application enhances travel convenience by integrating smart filters (such as time, airline, price, and number of stops), intuitive dashboards, and notifications via email or SMS. With a responsive web interface, mobile adaptability, and secure data handling, “**AERO CONNECT**” aims to modernize air travel bookings and bridge the gap between airlines and passengers.

b. Background:**i. Introduction of the Application**

AERO CONNECT is a next-generation flight reservation system designed to simplify and enhance the air travel booking experience. Developed in 2025, the application was born from the need for a smarter, faster, and more user-friendly way to search, compare, and book flights. Leveraging modern web technologies and intuitive design principles, **AERO CONNECT** bridges the gap between traveller’s and airline services through a digital platform that is both powerful and easy to use.

The application caters to a wide range of users including individual travellers, travel agents, and airline administrators. It brings together various components of the booking process—such as flight discovery, real-time availability, payment integration, and e-ticketing—into one cohesive

ecosystem. With its clean interface, dynamic features, and robust backend, **AERO CONNECT** aims to redefine how flight reservations are managed in today's fast-paced, mobile-driven world.

ii. Brief Note on Existing System

Traditional flight booking systems are often fragmented, outdated, and frustrating for users. Whether it's booking a seat, finding the best fare, or receiving timely updates, users frequently face multiple challenges:

1. **Cumbersome Interfaces:** Older systems tend to be complex and difficult to navigate, especially on mobile devices.
2. **Limited Customization:** Users are unable to filter results based on personal preferences such as budget, number of stops, or airline.
3. **Manual Updates & Delays:** Many platforms fail to reflect real-time changes in flight status or availability.
4. **Poor Communication:** Travelers rarely receive timely notifications about their bookings, leading to confusion or missed flights.
5. **Data Disorganization:** Airlines and agents often rely on multiple tools to manage schedules, passenger data, and payments, increasing the risk of errors.

AERO CONNECT addresses these gaps with a unified, streamlined solution that provides real-time data, smart filtering, secure transactions, and seamless communication for a truly modern booking experience.

c. Objectives of the System

The **AERO CONNECT** has been developed with a clear vision: to make flight booking faster, smarter, and more convenient for all users. The core objectives of the system are as follows:

1. Centralized Booking Platform

- Objective: To bring all flight reservation activities under a single, integrated platform.

- Benefit: Users can book, modify, and manage flights without switching between multiple systems.

2. Smart Search and Filtering

- Objective: To help users find flights that best meet their needs with flexible filters.
- Benefit: Saves time and improves satisfaction by showing relevant results based on user preferences.

3. Real-Time Availability and Pricing

- Objective: To provide up-to-the-minute updates on flight schedules and fares.
- Benefit: Reduces errors and ensures booking decisions are made with the latest information.

4. User-Friendly Interface

- Objective: To offer a clean, intuitive, and responsive design suitable for all devices.
- Benefit: Improves usability, especially for mobile users and frequent travelers.

5. Admin and Airline Management Tools

- Objective: To give administrators control over flight listings, fares, and customer records.
- Benefit: Enhances operational efficiency and reduces dependency on manual systems.

6. Instant Notifications and E-Tickets

- Objective: To automatically notify users of booking confirmations, changes, or cancellations.
- Benefit: Keeps travelers informed and reduces missed flights or delays.

7. Secure Booking and Payment

- Objective: To ensure transactions are processed safely with encryption and validation.

- Benefit: Protects user data and builds trust in the platform.

8. Scalable Architecture

- Objective: To support future growth in users, airlines, and destinations.
- Benefit: Ensures long-term performance and adaptability as user demand increases.

9. Analytics and Monitoring

- Objective: To track booking trends, user behavior, and system usage through real-time analytics.
- Benefit: Enables data-driven improvements and performance tuning.

d. Scope of the System

The **AERO CONNECT** is a comprehensive flight reservation application designed to serve multiple user types and streamline the booking process from start to finish. The key components within its scope include:

1. User Management

- **Scope:** Supports different user roles including travelers, agents, and airline staff.
- **Features:**
 - User registration and login
 - Role-based access control
 - Personal profile and booking history

2. Flight Search and Booking

- **Scope:** Allows users to search and book flights using a variety of filters.

Features:

- Search by destination, date, airline, stops
- Fare comparisons

- Seat selection and booking

3. Reservation and Payment Module

- **Scope:** Enables users to reserve and pay for flights securely.
- **Features:**
 - Multiple payment gateways
 - Secure transaction handling
 - Instant e-ticket generation

4. Flight and Fare Management (Admin Panel)

- **Scope:** Gives administrators control over flight schedules, pricing, and bookings.
- **Features:**
 - Add/edit/remove flights
 - Set fare rules and promotions
 - Manage cancellations and changes

5. Notification System

- **Scope:** Delivers real-time updates to users via SMS and email.
- **Features:**
 - Booking confirmation
 - Flight delay/cancellation alerts
 - Reminder notifications

6. Search & Filtering Capabilities

- **Scope:** Provides powerful tools to help users find the right flight.
- **Features:**
 - Keyword and destination-based search

7. Security and Data Protection

- **Scope:** Implements security measures to safeguard user data.
- **Features:**
 - Encrypted login and transactions
 - Compliance with global data protection standards

8. Mobile Responsiveness and Accessibility

- **Scope:** Ensures usability on desktops, tablets, and smartphones.
- **Features:**
 - Responsive UI/UX
 - Support for assistive technologies (e.g., screen readers)

9. Feedback and Support Tools

- **Scope:** Allows users to report issues and suggest improvements.
- **Features:**
 - Feedback forms and surveys
 - FAQ and live support options

e. Structure of the System

Designing the structure of a flight reservation system, like the **AERO CONNECT**, involves building a streamlined, scalable, and user-friendly platform that supports real-time flight search, secure bookings, and timely communication. Below is a high-level overview of the system architecture and its major components:

1. User Interface (UI)

- **Traveler Portal:** Enables users to search, book, and manage flights; view booking history; and receive updates.

- **Admin Portal:** Allows airline or app administrators to manage flight schedules, seat availability, pricing, and view user data.

2. Backend Services

- **Flight Management Service:** Manages flight listings, schedules, routes, fares, and real-time seat availability.
- **Booking & Ticketing Service:** Handles reservation logic, ticket generation, cancellations, and refund processes.
- **User Management Service:** Manages user authentication, profile data, roles, and login sessions.
- **Payment Integration Service:** Processes secure online payments using third-party gateways.
- **API Gateway:** Acts as a central hub for routing requests between the frontend interface and backend services securely.

3. Database

- **User Database:** Stores user credentials, personal details, and booking preferences.
- **Flight Database:** Contains comprehensive data on airlines, flights, seat inventory, and pricing.
- **Reservation Database:** Maintains booking records, passenger details, and ticketing history.
- **Transaction Database:** Stores payment details, invoices, and refund information securely.

4. Integration Services

- **Payment Gateway Integration:** Secure transactions through providers like Razorpay, Stripe, or PayPal.

- **Calendar Sync:** Allows users to export flight details to their personal calendars (Google, Outlook, etc.).
- **Flight Data APIs (Optional):** Integration with third-party airline APIs to fetch live data on flight availability and pricing

5. Security

- **Authentication:** Secure login with encrypted credentials and JWT-based session handling.
- **Authorization:** Role-based access control ensuring proper access for admins and users.
- **Encryption:** All personal and payment data is encrypted both in transit and at rest for maximum privacy.

7. Monitoring and Analytics

- **System Monitoring:** Logs and monitors app performance, usage patterns, and error reports.
- **Analytics Dashboard:** Provides insights into user behavior, booking trends, most popular routes, and transaction success rates.

Example System Flow

1. Admin Actions:

- The admin logs in to the portal.
- Adds or edits flight schedules, prices, and availability.
- Manages user and booking data securely.

2. Traveler Actions:

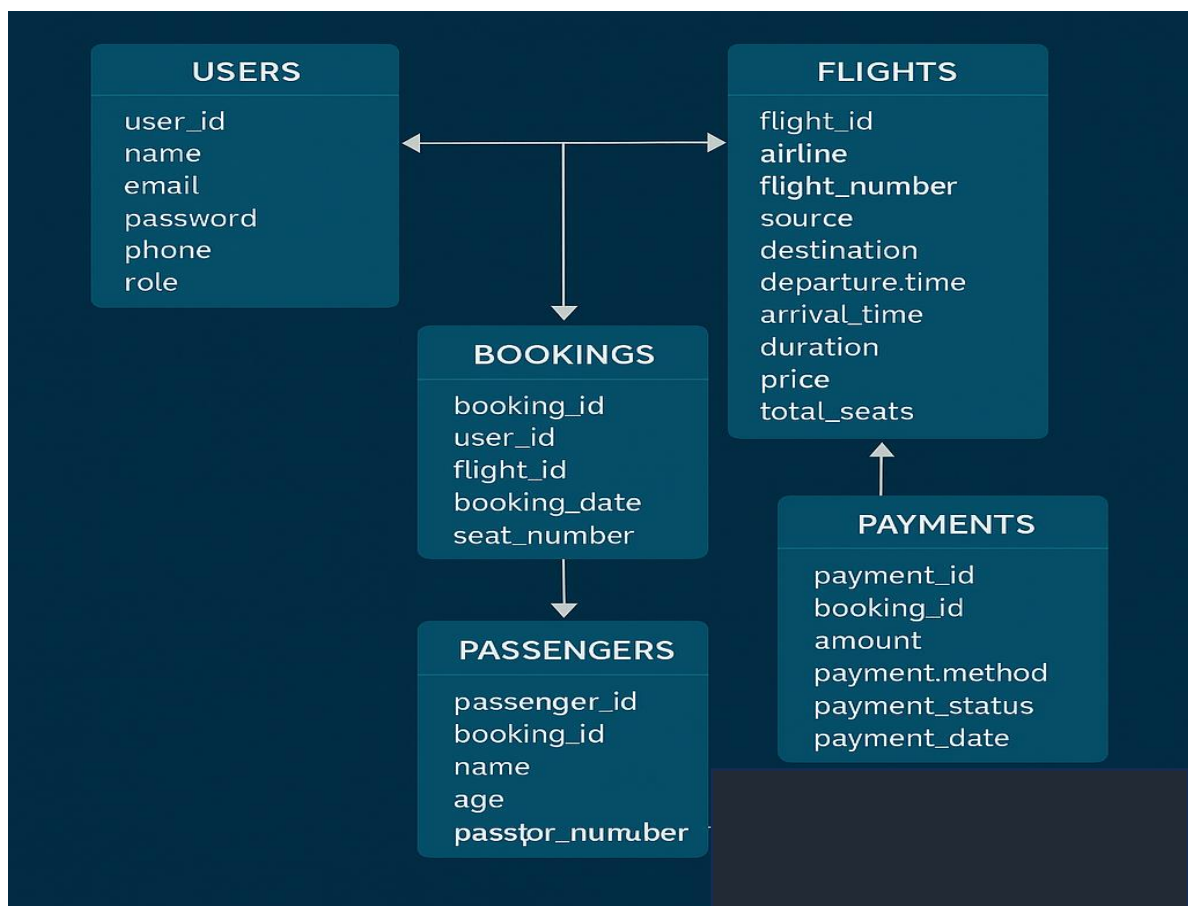
- The user logs in and searches for flights by route, date, or price.

- Books a preferred flight and completes payment.
- Receives a confirmation email and updates via push notifications or in-app alerts.

Technology Stack (Example)

- **Frontend:** HTML, CSS, JAVASCRIPT (for cross-platform mobile and web apps)
- **Backend:** Node.js, Express.js
- **Database:** MYSQL
- **Notification Services:** Firebase Cloud Messaging (FCM) for push notifications, Nodemailer package for sending email Notifications
- **Authentication:** Email Authentication,
- **Hosting:** AWS, Google Cloud Platform

Sample Database Schema



f. System Architecture for AERO CONNECT – Flight Reservation System

1. Frontend

- **Technologies:** HTML, CSS, JavaScript (React.js, Bootstrap)
- **Components:**
 - **User Portal:**
 - Dashboard: Displays upcoming flights, booking status, and payment history.
 - Search Flights: Allows users to search flights based on source, destination, dates, and number of passengers.
 - Flight Details: Shows detailed information about flights (departure/arrival time, price, seat availability).
 - Booking Management: Enables users to book and manage their flight reservations.
 - **Admin Portal:**
 - Dashboard: Displays flight booking statistics, user activities, and quick actions.
 - Flight Management: Interface for adding, editing, and removing flights.
 - Booking Management: View and manage all flight bookings.
 - User Management: Allows admins to manage user accounts and roles.

2. Backend

- **Technologies:** Node.js, Express.js, Cors.
- **Components:**

- **API Server:**
 - **User Management Service:**
 - Handles user registration, login, and profile management.
 - Provides APIs for user-related operations (login, registration, profile updates).
 - **Flight Service:**
 - Manages the creation, scheduling, and dispatch of flight details.
 - Provides APIs for searching, booking, and managing flights.
 - **Booking Service:**
 - Handles flight reservations, status updates, and cancellation requests.
 - Provides APIs for creating, retrieving, and managing bookings.
 - **Payment Service:**
 - Manages payment transactions for flight bookings.
 - Integrates with payment gateways (e.g., Stripe, PayPal).
 - **Notification Service:**
 - Sends confirmation emails, booking alerts, and reminders.
 - Integrates with email, SMS, and push notification platforms.

3. Database

- **Technology:** MySQL
- **Schema:**
 - **User Table:**
 - user_id: UUID, Primary Key
 - name: VARCHAR
 - email: VARCHAR
 - password hash: VARCHAR
 - birthday: DATE
 - gender: VARCHAR
 - phone: VARCHAR
 - created_at: DATETIME
 - **Flight Table:**
 - flight_id: UUID, Primary Key
 - airline name: VARCHAR
 - departure time: DATETIME
 - arrival time: DATETIME
 - price: DECIMAL
 - available seats: INT
 - **Booking Table:**
 - booking: Primary Key
 - user_id: UUID (Foreign Key to User Table)
 - seat number: VARCHAR
 - booking status: ENUM ('pending', 'confirmed', 'cancelled')

- booking date: DATETIME
- Luggage: VARCHAR
- Date: DATE
- Destination: VARCHAR
- Travel_class: VARCHAR

4. Integration Services

- **Components:**

- **Calendar Integration:**

- Syncs flight booking details with users' calendars for reminders about departure times.

- **Payment Gateway Integration:**

- Connects to external payment platforms (e.g., Stripe, PayPal) for secure processing of payments.

- **Third-Party Integration:**

- Connects with other systems for additional services (e.g., loyalty programs, travel insurance).

5. Security

- **Components:**

- **Authentication:**

- Uses JWT (JSON Web Tokens) for secure authentication and session management.
 - Multi-factor authentication (MFA) can be added for extra security.

- **Authorization:**

- Role-based access control for different types of users (users, admins).

- **Encryption:**

- Uses SSL/TLS encryption to protect sensitive data in transit (HTTPS).
- Sensitive user and payment data is encrypted at rest in the database.

6. Monitoring and Analytics

- **Components:**

- **Logging and Monitoring:**

- Tracks system performance, user activities, and errors using tools like Winston for logging and Prometheus for monitoring.

- **Analytics:**

- Provides insights into flight bookings, user engagement, payment trends, and system usage.
- Integrates with platforms like Google Analytics or custom-built dashboards for real-time insights.

Example System Flow

1. Admin Actions:

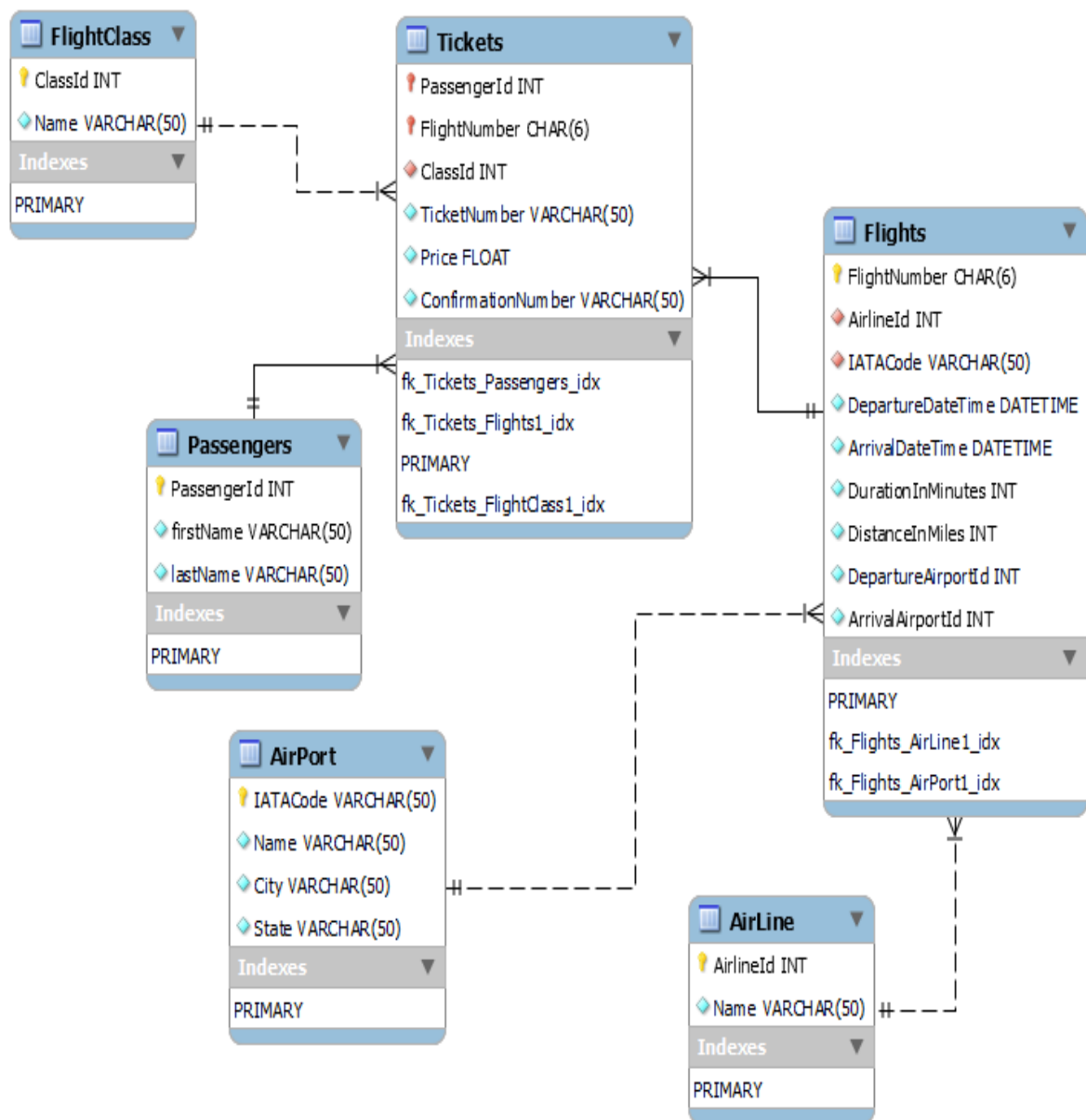
- Admin logs into the admin portal.
- Admin adds or updates flight details (airline, route, schedule, price).
- Admin manages user bookings (view, confirm, cancel).
- Admin monitors system usage and notifications.

2. User Actions:

- Users log into their portal to view available flights.

- Users search for flights based on source, destination, dates, and passenger details.
- Users proceed to book flights, confirm payment, and receive booking confirmations via email/SMS.
- Users can view, modify, or cancel bookings from the user portal

ARCHITECTURE OF THE SYSTEM



End Users:

The “AERO CONNECT” caters to various types of users, each with distinct roles and functionalities:

1. Passengers (Users)

- **Role:** Primary recipients of flight reservation notifications.
- **Functionalities:**
 - Search for flights, view details, and book reservations.
 - Receive notifications about flight status, bookings, cancellations, and offers.
 - Set and manage notification preferences (email, SMS, push notifications).
 - View flight booking history and status.
 - Access boarding pass and ticket information.
 - Participate in surveys or feedback regarding their flight experience.

2. Airline Administrators

- **Role:** Manage flight schedules, bookings, and system-wide notifications.
- **Functionalities:**
 - Create and manage flight schedules and availability.
 - Approve, modify, or cancel flight bookings.
 - Manage users (passengers) and view booking histories.
 - Send notifications to passengers regarding flight delays, cancellations, and promotions.
 - View and analyse system reports such as flight occupancy, revenue, and user engagement.

3. Customer Support Representatives

- **Role:** Handle user inquiries, booking issues, and complaints.
- **Functionalities:**
 - Assist passengers with booking issues, flight inquiries, and payment problems.
 - Manage cancellation requests and refund processes.
 - Communicate with passengers through in-app messages, email, or phone calls.

Example Use Cases

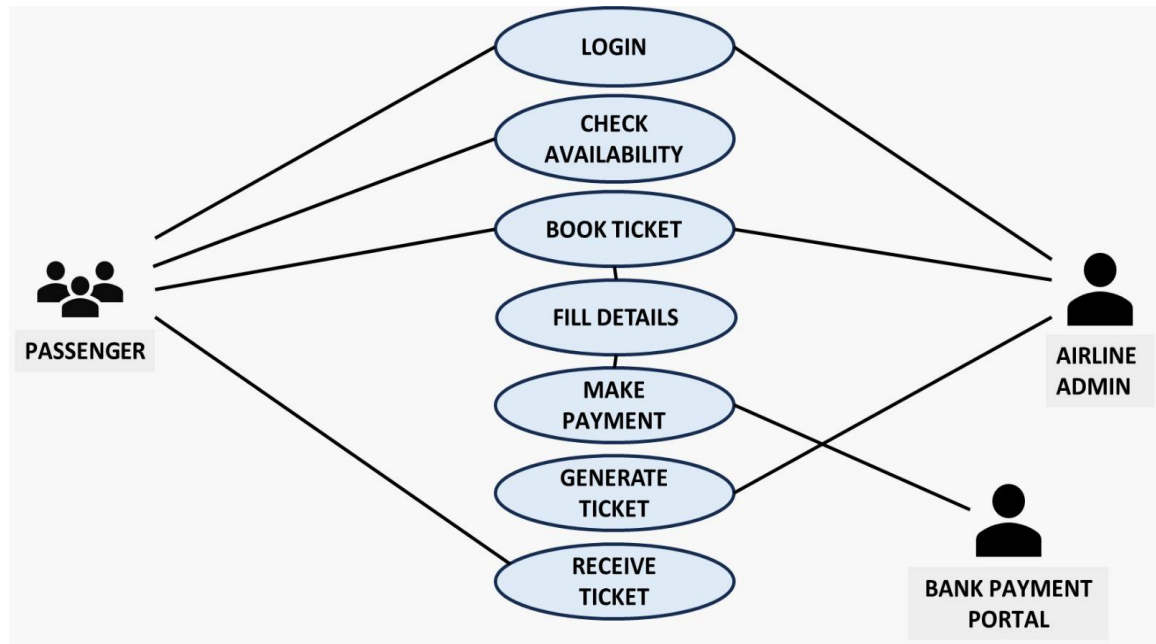
1. Passenger Use Case:

- **Scenario:** Booking a Flight and Receiving Updates
 - The passenger logs into the app, searches for available flights, and books a ticket.
 - The system sends a confirmation email and a push notification to the passenger.
 - A few hours before the flight, the passenger receives a push notification with flight details and check-in instructions.
 - The passenger receives a final notification when the flight is boarding.

2. Administrator Use Case:

- **Scenario:** Managing Flight Schedules and Sending Notifications
 - The administrator logs into the admin portal.
 - Creates a new flight schedule, sets pricing, and availability.
 - Sends a notification to passengers regarding a change in the flight schedule.

- The administrator later reviews the engagement analytics of sent notifications to monitor how many passengers received or engaged with the notification.



3. Customer Support Use Case:

- **Scenario:** Handling a Flight Cancellation
 - A passenger contacts support about a cancelled flight.
 - The customer support representative reviews the booking and initiates a refund process.
 - They send a follow-up notification to the passenger, confirming the refund and providing alternative flight options.

h. Software and Hardware Used for Development

1. Software

- **Development Environment:**
 - **IDE/Text Editors:** Visual Studio Code.

- **Node.js and Express.js:** For backend development
- **Frontend Technologies:** HTML, CSS, JavaScript, react for the web app; React Native for mobile apps
- **Database:** MySQL for storing user and flight data
- **Authentication:** JWT (JSON Web Tokens)
- **API Documentation:** Postman
- **Frameworks and Libraries:**
 - **Frontend:**
 - **CSS Framework:** Bootstrap, Tailwind CSS
 - **JavaScript Libraries:** React.js, Axios for API calls
 - **Backend:**
 - **Express.js:** Web framework for Node.js
 - **Nodemailer:** For sending emails
 - **Winston:** Logging library
- **DevOps:**
 - **Containerization:** Docker for isolated development environments
 - **CI/CD:** Jenkins, GitHub Actions for continuous integration and deployment
 - **Hosting:** AWS, Google Cloud, Heroku for cloud-based hosting
 - **Monitoring:** Prometheus, Grafana for real-time monitoring and analytics

2. Hardware

- **Development Machines:**
 - **Minimum Specifications:** Dual-core CPU, 8GB RAM, 256GB SSD

- **Recommended Specifications:** Quad-core CPU, 16GB RAM, 512GB SSD
- **Servers:**
 - **Development and Testing:** Cloud-based development environments (AWS EC2, Google Cloud VM)
 - **Production:** Cloud-based servers with SSD storage for better performance

System Architecture

The “AERO CONNECT” is designed with scalability, security, and performance in mind. Here’s an overview of its architecture:

1. Frontend (User Interface):

- **Passenger Portal:** Allows passengers to search for flights, book tickets, and view notifications.
- **Admin Portal:** Provides administrators with tools to manage flight schedules, bookings, and notifications.

2. Backend (API Server):

- **User Management Service:** Handles registration, login, profile management, and authentication (JWT).
- **Flight Service:** Manages flight schedules, availability, and details.
- **Booking Service:** Handles flight booking, payment processing, and ticket generation.
- **Notification Service:** Sends flight-related notifications via email, SMS, and push notifications.

3. Database (Data Storage):

- **User Table:** Stores passenger information, preferences, and login details.
- **Flight Table:** Stores flight schedules, details, and availability.

- Booking Table: Tracks flight bookings, payment status, and history.
- Payment Table: Stores payment transactions and status.

CHAPTER-2

Software Requirements Specification

a. Introduction

1. Introduction

The **Software Requirements Specification (SRS)** for the **AERO CONNECT: Flight Reservation System** outlines the essential functionalities, constraints, and interactions of the proposed web-based flight booking system. This document serves as a foundational guide for developers, project managers, and stakeholders, ensuring alignment in the vision, development, and deployment of the application.

AERO CONNECT is a dynamic and user-centric web application designed to simplify the flight reservation process for passengers while streamlining administrative operations for airline staff. Built using the robust and scalable **MERN stack**, this application is designed with responsiveness, performance, and ease of use at its core.

This document details both the **functional** and **non-functional** requirements, as well as the interfaces, user characteristics, design constraints, and performance expectations to ensure the successful implementation and maintenance of the system.

b. Overall Description

i. Product Perspective

The **AERO CONNECT** is a standalone, responsive web-based flight reservation system that seamlessly integrates with payment gateways and airline databases. It is accessible to users via any modern web browser and is designed to support both desktop and mobile users.

ii. Product Functions

Passenger Functions:

- User registration and authentication
- Search for available flights based on filters (date, source, destination, airline, price)
- View flight details

- Book flights and receive confirmation
- Manage bookings (cancel or modify)
- Make secure payments
- Receive notifications via email or SMS
- View travel history and digital boarding passes

Admin Functions:

- Admin login and secure access
- Add, update, or delete flight schedules
- Manage bookings and monitor system analytics
- Handle user accounts and customer service queries
- Access reporting dashboards

iii. User Characteristics

- **Passengers:** General public with basic knowledge of using web/mobile applications
- **Admin Users:** Airline staff with knowledge of airline operations and basic technical proficiency

iv. General Constraints

- Web application must be cross-browser compatible (Chrome, Firefox, Safari, Edge)
- Must support a high number of concurrent users
- Real-time database updates must be synchronized
- Data security and user privacy must be ensured at all times

v. Assumptions

- Users have internet access and mobile/desktop devices
- Payment gateways (e.g., Razor pay/Stripe) are configured
- Admin users have the necessary credentials and training

c. Functional Requirements

i. User Management Module

- **Passenger Registration/Login:** Users can create and access accounts
- **Profile Management:** Update personal details, manage saved preferences
- **Admin Login:** Secure access to admin dashboard

ii. Flight Search and Booking Module

- **Search Flights:** Filter by origin, destination, date, price, etc.
- **Book Flight:** Select and reserve flight seats
- **Payment Gateway:** Secure transaction through integrated API
- **Booking Confirmation:** print/download confirmation and digital ticket generation
- **Cancel/Modify Booking:** Allow users to alter or cancel their bookings

d. Design Constraints

- Backend: Node.js + Express.js
- Frontend: React.js with Bootstrap/Tailwind for responsive UI
- Database: MySQL
- RESTful API architecture
- Must follow mobile-first and accessibility design principles

e. System Attributes

- **Reliability:** Minimal downtime with cloud-hosted deployment
- **Availability:** Accessible 24/7, including mobile and desktop support
- **Security:** Uses JWT for user authentication and HTTPS encryption
- **Maintainability:** Modular MERN stack allows easy updates and debugging

- **Performance:** Optimized queries and load balancing for scalability
- **Usability:** Clean, intuitive interface with visual flight cards, filters, and prompts

f. Other Requirements

- **Backup and Recovery:** Daily database backups and disaster recovery strategy
- **Scalability:** Horizontal scaling supported using cloud providers (e.g., AWS, GCP)
- **Monitoring:** Use of tools like Prometheus and Grafana for system performance and alerting

CHAPTER-3

System Design (Functional Design)

a. Introduction

System design is a vital stage in the software development lifecycle that defines the architecture, modules, data flow, and interface interactions needed to fulfil system requirements. For the AERO CONNECT the system design phase transforms the Software Requirements Specification (SRS) into a tangible and implementable framework.

The primary goal of the system design for AERO CONNECT is to lay down a detailed blueprint for developers that ensures the application is efficient, scalable, secure, and user-friendly. The application is crafted using the robust and modern MERN stack—MySQL, Express.js, React.js, and Node.js—which ensures a seamless experience for both users and administrators in the flight booking ecosystem.

The system design process is divided into:

- **High-Level Design (HLD):** Describes the system's overall structure, key modules, and their interactions.
- **Low-Level Design (LLD):** Delves into the internal logic, algorithms, and data structures of each module.

Key objectives include:

- **Clarity:** Clear understanding of system behaviour and architecture for all stakeholders.
- **Modularity:** Division into manageable and reusable components.
- **Interoperability:** Defined interfaces for smooth interaction between components.
- **Performance:** Optimized architecture for high responsiveness and throughput.
- **Security:** Built-in mechanisms for data protection and secure access.
- **Usability:** Intuitive design for an optimal user experience across devices.

b. Assumptions and Constraints

Assumptions

1. **User Access:** Users (passengers and admins) will use modern web browsers with internet connectivity.
2. **Infrastructure:** Cloud-based servers or hosting platforms will be available for deployment (e.g., Vercel, Render, or AWS).
3. **Payment Integration:** A secure third-party payment gateway (e.g., Stripe, Razor pays) will be available.
4. **User Accuracy:** Information entered by users is assumed to be accurate (e.g., name, passport ID).
5. **Admin Proficiency:** Admins have basic technical knowledge to manage flight and booking data.
6. **Growth Readiness:** The system is expected to handle increasing traffic as user adoption grows.
7. **Regular Maintenance:** Scheduled maintenance and updates will be conducted.

Constraints

1. **Technology Stack:** MERN stack must be used (MySQL, Express.js, React.js, Node.js).
2. **Cross-Browser Compatibility:** Must work seamlessly on Chrome, Firefox, Safari, and Edge.
3. **Concurrent Users:** System must support large-scale user interactions simultaneously.
4. **Security Compliance:** Must include authentication (JWT) and encryption (HTTPS, bcrypt).
5. **Regulatory Compliance:** Must adhere to data privacy laws (e.g., GDPR, PCI-DSS).
6. **Performance Metrics:** Fast page load time, API response < 200ms under load.
7. **Accessibility:** Must conform to WCAG guidelines for inclusive design.
8. **Data Backup:** Automated daily backups and disaster recovery must be ensured.

c. Functional Decomposition

Breaking the system into functional components enables modular development and simplifies understanding.

1. User Management

1. Registration & Authentication

- User Sign-up (Passengers)
- Email Verification
- Login/Logout

2. Profile Management

- View/Edit Personal Information
- Manage Password

3. Admin Access Control

- Admin Login
- Role-Based Access (Admin/Passenger)

2. Flight Management

1. Flight Scheduling

- Add/Edit/Delete Flight Details
- Set Flight Status (Active/Cancelled)

2. Flight Search & Filters

- Filter by Date, Source, Destination
- Sort by Duration, Price, Stops

3. Booking Management

1. Ticket Booking

- Choose Flight & Seats
- Apply Offers/Promo Codes

2. Payment Integration

- Secure Payment via UPI
- Secure payment via wallet
- Generate Digital Ticket

3. Booking History

- View Past & Upcoming Trips
- Cancel/Modify Bookings

5. System Monitoring & Reports

1. Admin Dashboard

- Manage Flights & Users
- View System Logs & Activity

2. Reporting

- Booking Analytics
- Revenue and Usage Reports

I. Context Flow Diagram (CFD)

A Context Flow Diagram (or Level-0 DFD) shows the overall flow of data between the AERO CONNECT and external entities. Here's an overview of the interactions:

External Entities:

- Passenger
- Admin
- Payment Gateway (Stripe/Razor pay)
- Email/SMS Services

System (AERO CONNECT)

- Receives input from passengers/admins
- Processes flight searches and bookings
- Sends data to the database
- Communicates with payment APIs
- Dispatches email/SMS notifications

Flow Diagram

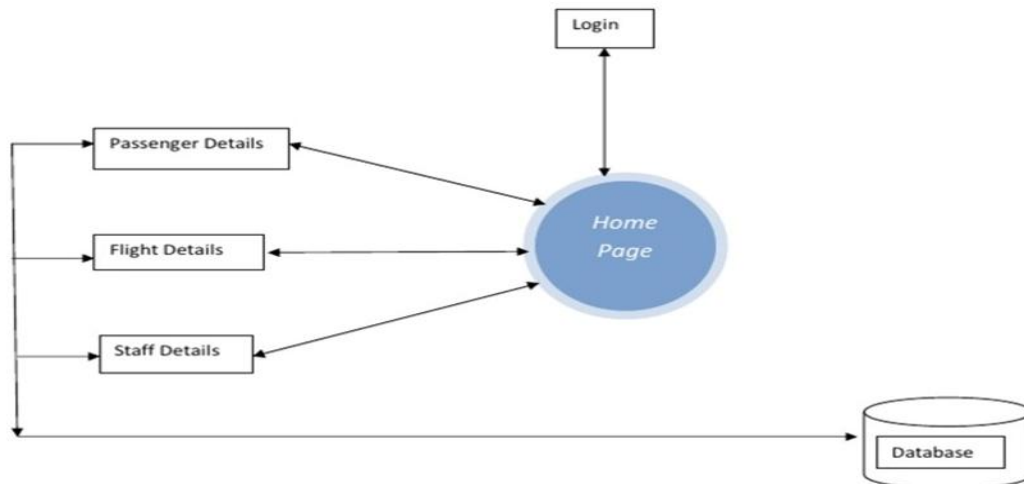


ii. Data Flow Diagrams (DFDs)

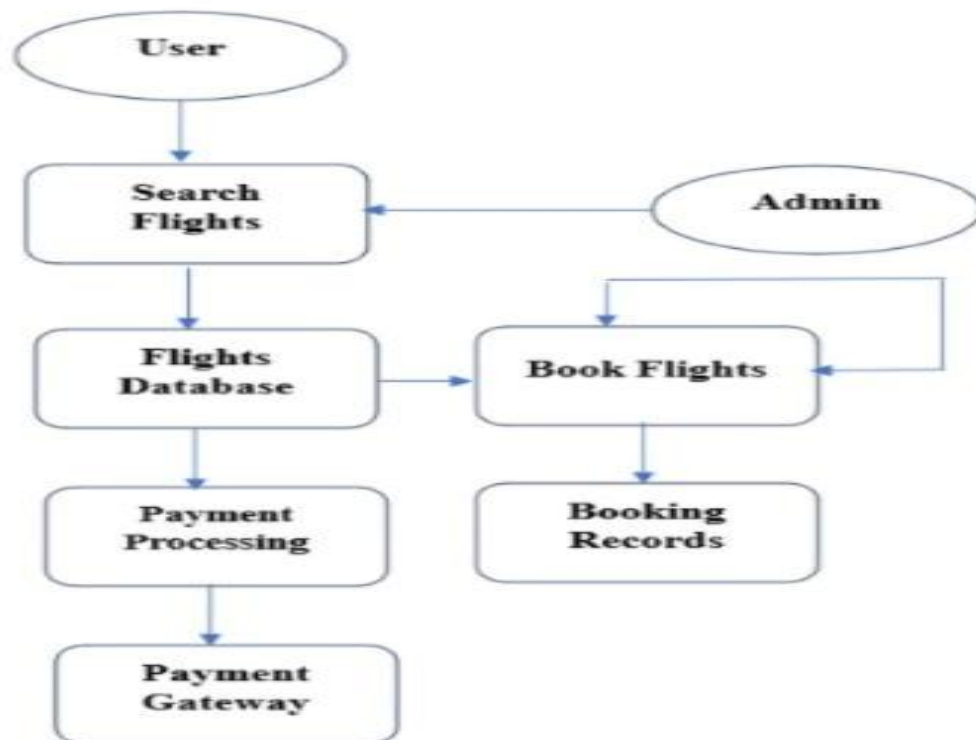
Data Flow Diagrams (DFDs) provide a detailed representation of the system. They are divided into multiple levels to show increasing detail.

Level 0 DFD:

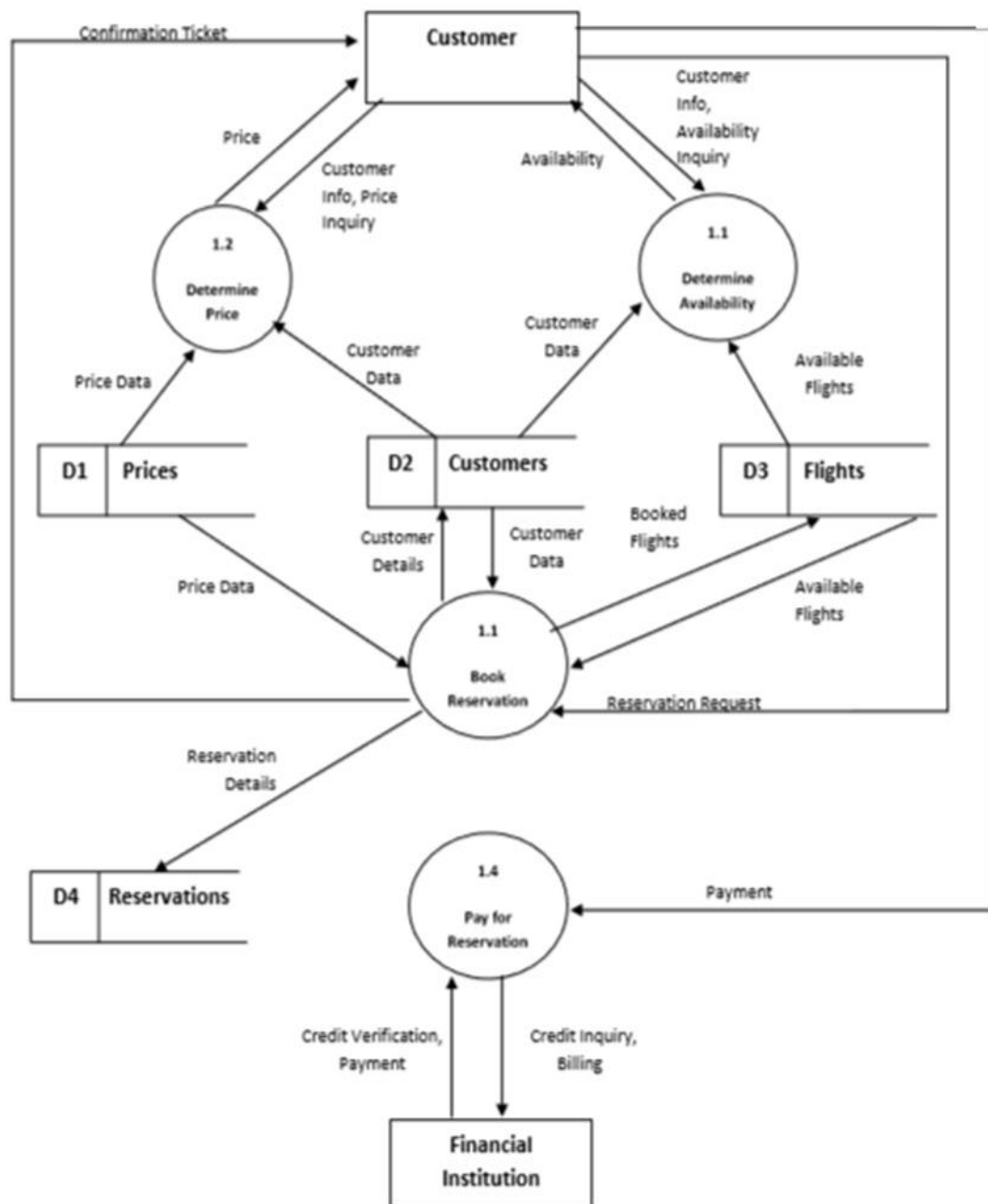
The **Level 0 DFD** provides an **overview of the Airline Reservation System**, showing the main processes and the interactions with external entities.



Level 1 DFD:



Level 2 DFD:



e. Description of Components

i. Functional Component

1: User Management (Login/Logout)

Description:

This component is the gateway to the **AERO CONNECT**, managing secure access for users including passengers and system administrators. It handles **user authentication and session management**, ensuring that access to features such as flight booking, profile management, and administration tools is protected and role-specific.

Features:

- **User Login:** Authenticates users using their registered credentials.
- **User Logout:** Ends active sessions and clears tokens.
- **Session Management:** Uses tokens (JWT) to manage user sessions and ensure secure, personalized access.

Technologies Used:

- **Frontend:** React.js – Responsive user interface for login/logout pages.
- **Backend:** Node.js with Express.js – API logic for authentication.
- **Database:** MongoDB – Stores encrypted user data and roles.
- **Security:** Implements bcrypt for password hashing, JWT for token-based sessions, and HTTPS for secure communication.

Workflow:

1. Login Process:

- User enters email and password on the login screen.
- React.js sends a POST request to the backend API (/api/login).

- Express.js controller validates credentials using MySQL.
- On success, a JWT token is issued and stored in local Storage.
- User is redirected to the dashboard (passenger/admin view).

2. Logout Process:

- User clicks the logout button.
- React.js clears the session token and redirects to the login page.
- Backend can invalidate the token for enhanced security.

ii. Functional Component

2: Flight & Booking Management

Description:

This is the core engine of the **AERO CONNECT**, managing flight schedules, seat availability, and booking processes. It allows admins to manage flight information and passengers to search, book, and manage their reservations with ease.

Features:

- **Flight Management:** Admins can add, update, or cancel flights.
- **Flight Search:** Users can search flights based on date, origin, destination, and airline.
- **Seat Booking:** Passengers can select available seats and confirm their bookings.
- **Ticket Generation:** A digital e-ticket is generated after successful booking.
- **Booking History:** Users can view or cancel past and upcoming bookings.

Technologies Used:

- **Frontend:** React.js – Interactive UI for flight selection and booking process.
- **Backend:** Node.js with Express.js – RESTful APIs for flight and booking logic.

- **Database:** MongoDB – Stores flight schedules, seat maps, and booking records.
- **Payment Integration:** (e.g., Stripe/Razor pay) – For secure payment processing.

Workflow:

1. Flight Creation (Admin):

- Admin logs in and navigates to the Flight Management Panel.
- Admin fills flight details and submits via React form.
- Backend API saves flight info in MySQL

2. Flight Search & Booking (Passenger):

- User selects travel details (origin, destination, date).
- React sends query to backend (/api/flights/search).
- Matching flights are displayed; user selects one.
- Seat selection and passenger details are submitted.
- Payment is processed via integrated gateway.
- Booking is confirmed and an e-ticket is issued.

3. Booking History and Cancellation:

- User views all bookings from their profile dashboard.
- Option to cancel upcoming trips is available.
- On cancellation, backend updates status and refunds if applicable.

User Interface Overview:

- **Login Page:** Secure access point with email/password fields.
- **Passenger Dashboard:** Displays available flights, upcoming trips, and booking history.
- **Admin Dashboard:** Tools to manage flights, view bookings, and monitor usage.
- **Flight Search Page:** Filters and search results for finding flights.
- **Booking Page:** Intuitive seat selection and payment interface.
- **Ticket Confirmation Page:** Displays digital ticket and itinerary

CHAPTER-4

Database Design

a. Introduction

Database Design

The **AERO CONNECT** database is meticulously designed to ensure smooth and efficient handling of **user data, flight schedules, bookings, and payment transactions**. The primary objective of the database design is to uphold **data integrity, security, and fast access** while supporting all key operations of the flight reservation system.

This application leverages **MySQL**, a **NoSQL document-based database**, which offers flexibility, scalability, and seamless integration with other components of the **MERN stack**. The database is structured to efficiently manage entities such as **users, flights, bookings, payments, and seat availability**, while enabling rapid queries and secure storage.

b. Purpose and Scope

Purpose:

The **purpose** of the database within the **AERO CONNECT** is to serve as the central data repository that powers all reservation system functionalities. From managing user accounts and flight schedules to processing seat reservations and tracking payment statuses, the database plays a crucial role in delivering a reliable and responsive booking experience.

The design ensures that:

- **Passengers** can easily book, view, and manage their trips.
- **Admins** can manage flights, monitor bookings, and generate reports.
- **Data** flows seamlessly and securely across all components.

Scope:

The scope of the database covers the following key areas:

- **User Authentication & Authorization:**
 - Stores user credentials with hashed passwords.
 - Manages roles (e.g., passenger, admin) and permissions.
- **Flight Management:**
 - Stores data on available flights, including departure/arrival times, dates, airlines, routes, and seat configurations.
- **Booking Management:**
 - Tracks passenger bookings, assigned seats, booking status, and timestamps.
 - Links each booking with specific users and flights.
- **Payment Handling:**
 - Records payment confirmations and transaction references.
 - Ensures booking confirmation only after successful payment.
- **Seat Availability & Allocation:**
 - Dynamically updates seat availability to prevent double booking.
- **Cancellation and Refund Records:**
 - Maintains cancellation logs and refund eligibility.
- **System Security:**
 - Implements role-based access control.
 - Ensures sensitive data is encrypted and protected from unauthorized access.

b. Table Definition

In the **AERO CONNECT**, the database schema is crafted to ensure efficient data storage, rapid access, and integrity of records related to **users, flights, bookings, and payments**. Though the system uses **MySQL**, for clarity, the structure below is

represented in a relational format (like SQL tables), mapping directly to **MySQL collections** with similar field types and constraints.

Tables

1. Users Table

Field	Type	Null	Key
id	INT	NO	PRIMARY KEY, AUTO_INCREMENT
Name	VARCHAR(100)	NO	
email	VARCHAR(255)	NO	UNIQUE
password	VARCHAR(255)	NO	
phone	VARCHAR(20)	NO	UNIQUE
birthday	DATE	YES	
gender	VARCHAR(10)	YES	
Created_at	DATETIME	YES	

2. Flights Table

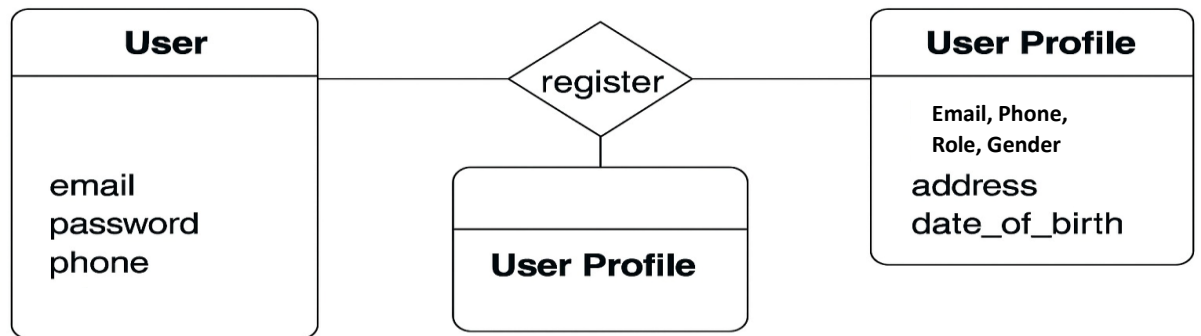
Field	Type	Null	Key
id	INT	NO	PRIMARY KEY, AUTO_INCREMENT
airline	VARCHAR(100)	NO	
Flight_number	VARCHAR(50)	NO	UNIQUE
Depature_city	VARCHAR(100)	NO	
Arrival_city	VARCHAR(100)	NO	
Departure_time	DATETIME	NO	
Arrival_time	DATETIME	NO	
price	DECIMAL(10,2)	NO	
Seats_available	INT	NO	

Bookings Table

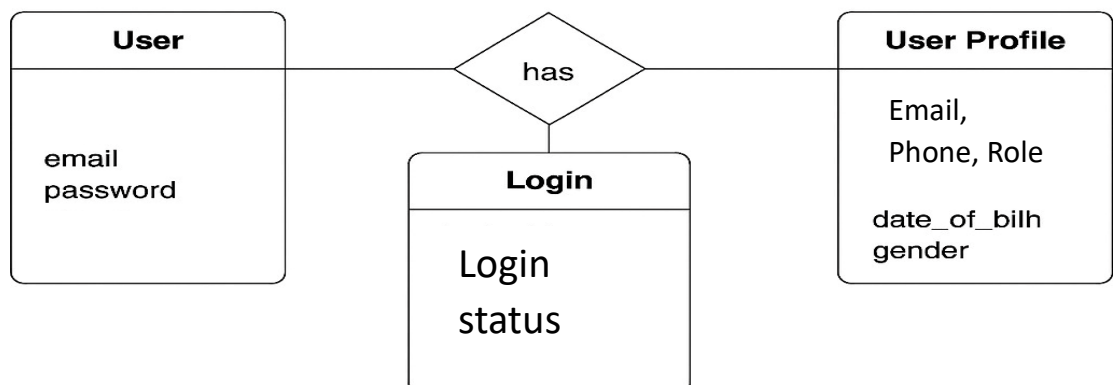
Field	Type	Null	Key
id	INT	NO	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(100)	NO	
email	VARCHAR(100)	NO	UNIQUE
phone	VARCHAR(20)	NO	UNIQUE
seat	VARCHAR(10)	YES	
meal	VARCHAR(50)	YES	
luggage	VARCHAR(50)	YES	
date	DATE	NO	
Flight_number	VARCHAR(50)	NO	
airline	VARCHAR(100)	NO	
Source	VARCHAR(100)	NO	
destination	VARCHAR(100)	NO	
Travel_class	VARCHAR(50)	NO	

c. ER diagrams :

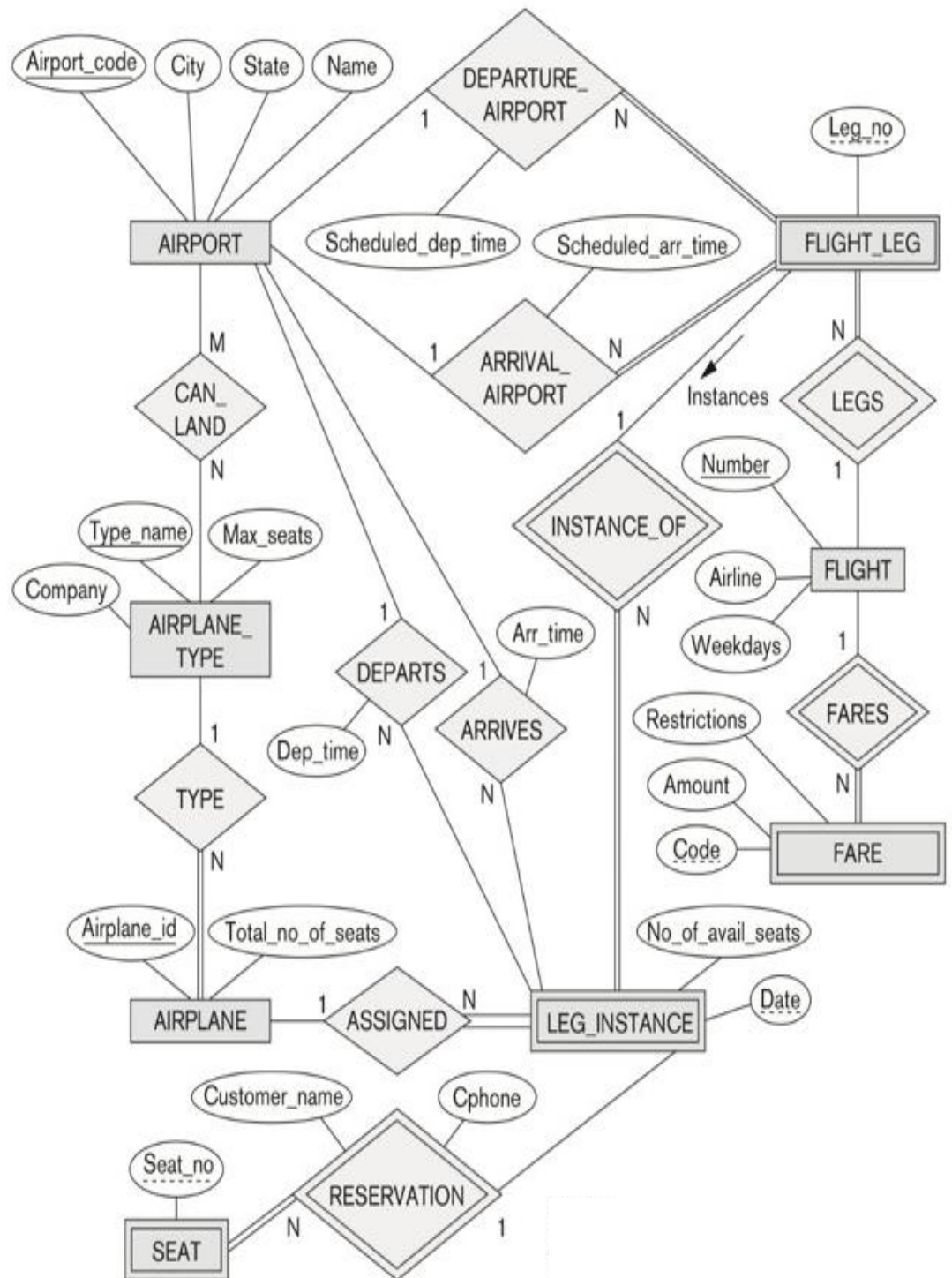
1.Registration E-R diagram



2.Login E-R diagram



3. Flight Booking Site E-R diagram



CHAPTER-5

Detailed Design (Logic design of modules)

a. Introduction

Database Design

The database design for the “**AERO CONNECT**” **Flight Reservation System** is thoughtfully engineered to enable smooth, secure, and scalable management of user data, flight schedules, bookings, and payments. Built using the **MERN stack**, the system uses **MySQL**, a powerful NoSQL database, to store and manage data in a flexible, document-oriented structure.

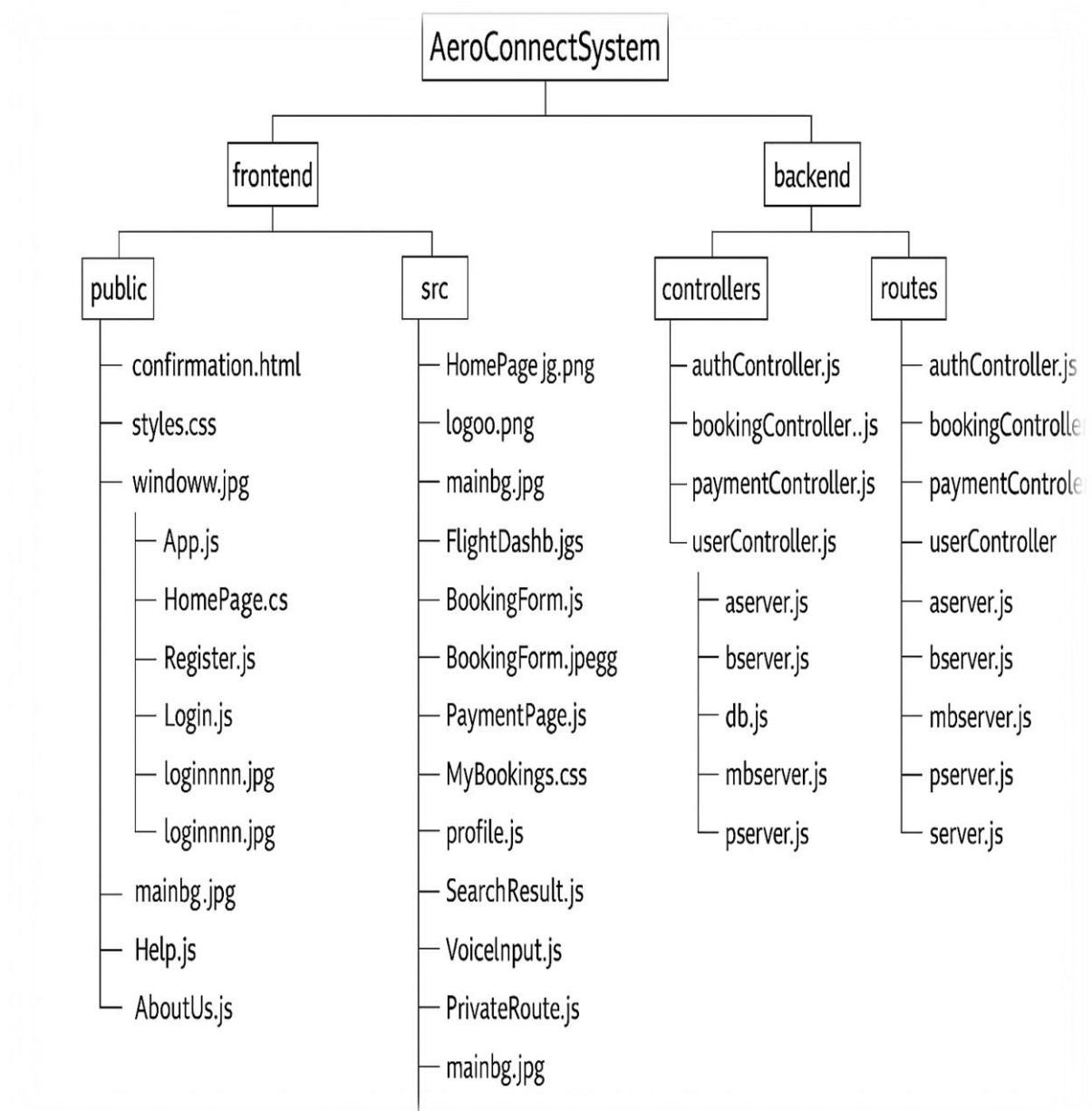
This design ensures high performance, seamless data access, and strong data integrity without sacrificing scalability. The application's core entities—such as **users, flights, bookings, and payment transactions**—are efficiently modeled using collections that mirror real-world processes, making the system intuitive and responsive.

To ensure secure handling of sensitive information like user credentials and payment records, best practices such as **encryption, token-based authentication (JWT), and password hashing (bcrypt)** are implemented. Relationships between data are managed using embedded documents and references, optimizing performance while maintaining logical connections across collections.

The **AERO CONNECT** database design not only supports current features like **flight search, real-time booking, user authentication, and booking history tracking**, but also provides the flexibility for **future expansion**—including features like loyalty programs, personalized recommendations, and admin analytics dashboards.

By leveraging the modern capabilities of **MySQL** along with the full power of **Node.js and Express** on the server side, the app ensures that passengers, administrators, and operators experience **fast, reliable, and secure interactions** with the system.

b. Structure of the Software Package (Structure Chart)



c. Modular Decomposition of the System

i. Module 1: User Authentication (Login/Logout)

The **User Authentication** module of the **AERO CONNECT** ensures secure and streamlined access for all users—passengers, admins, and airline operators. It plays a critical role in safeguarding user data while maintaining a smooth entry point into the flight booking experience. Using industry-standard practices like **JWT (JSON Web Token)** authentication and **bcrypt password hashing**, this module is designed for both **security** and **performance**.

1. Inputs

- **Login:**
 - username (String)
 - password (String)
- **Logout:**
 - session token (String)

2.Procedural Details

Login Workflow

1. The user enters their **email/username** and **password** into the login form on the frontend (React.js).
2. React.js captures the credentials and sends them via a **POST** request to the `/api/auth/login` endpoint.
3. The backend (Node.js + Express) receives the credentials and queries the **MySQL** database for the matching user document.
4. If a match is found, the backend compares the entered password with the **hashed password** stored in the database using **bcrypt**.
5. On a successful match, a **JWT session token** is generated.

6. The token is returned to the frontend, which stores it (typically in local storage or cookies).
7. The user is redirected to the appropriate dashboard based on their role (e.g., Passenger, Admin).

Logout Workflow

1. The user initiates a logout request by clicking the logout button.
2. The frontend sends a **POST** request to the `/api/auth/logout` endpoint with the **session token** in the headers.
3. The backend verifies the token and marks it as invalid (if using a token blacklist or refresh token logic).
4. The frontend clears the session data and redirects the user to the **login page**.

3. File I/O Interfaces

- **Login:**

- **Input:**

- JSON request body: {"username": "...", "password": "..."}

- **Output:**

- JSON response: {"token": "...", "role": "Passenger", "message": "Login successful"}

- **Logout:**

- **Input:**

- Request header: Authorization: Bearer <token>

- **Output:**

- JSON response: {"message": "Logout successful"} or error

4. Outputs

- **Login:**
 - **Success:**
 - Valid session token, user role, redirection to dashboard
 - **Failure:**
 - Error message (e.g., “Invalid credentials”)
- **Logout:**
 - **Success:**
 - Logout confirmation, redirection to login
 - **Failure:**
 - Error message (e.g., “Session invalid or expired”)

5. Implementation Aspects

Frontend (React.js):

- **Components:**
 - Login.js: Handles user input, form validation, and sends login request.
 - Logout.js: Sends logout request and clears session data.
- **State Management:**
 - Session token and user role are stored in **React Context**, **Redux**, or **local Storage**.
- **Routing:**
 - React Router is used to **redirect** authenticated users to dashboards and restrict access to protected routes.

Backend (Node.js + Express):

- **Controllers:**

- `authController.js`:
 - `login ()`: Validates credentials, generates JWT token.
 - `logout ()`: Optionally blacklists or expires the session token.

- **Routes:**

- Defined in `authRoutes.js` for `/login` and `/logout` endpoints.

- **Security:**

- **Password Hashing:** Uses `bcrypt` to securely hash and verify passwords.
- **Session Management:** Implements **JWT-based authentication** with token expiration and refresh mechanisms.

CHAPTER-6

Program Code Listing

a. Database Connection in Node.js

To connect a Node.js application to a MySQL database, you typically use a MySQL client for Node.js, such as mysql2 or sequelize. Below, I'll provide examples of how to set up a database connection using both mysql2 and sequelize.

Install mysql2:

Create a database configuration file

```
1 • CREATE DATABASE IF NOT EXISTS flight_booking;
2 • USE flight_booking;
3
4 -- USERS TABLE
5 • CREATE TABLE IF NOT EXISTS users (
6     id INT AUTO_INCREMENT PRIMARY KEY,
7     username VARCHAR(100) NOT NULL UNIQUE,
8     email VARCHAR(100) NOT NULL UNIQUE,
9     password VARCHAR(255) NOT NULL,
10    role ENUM('admin', 'customer') DEFAULT 'customer',
11    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
12 ) ENGINE=InnoDB;
13
14 -- FLIGHTS TABLE
15 • CREATE TABLE IF NOT EXISTS flights (
16     id INT AUTO_INCREMENT PRIMARY KEY,
17     airline VARCHAR(100) NOT NULL,
18     flight_number VARCHAR(50) NOT NULL UNIQUE,
19     departure_city VARCHAR(100) NOT NULL,
20     arrival_city VARCHAR(100) NOT NULL,
21     departure_time DATETIME NOT NULL,
22     arrival_time DATETIME NOT NULL,
23     price DECIMAL(10, 2) NOT NULL,
24     seats_available INT NOT NULL
25 ) ENGINE=InnoDB;
26
```

```

CREATE TABLE IF NOT EXISTS bookings (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL,
  phone VARCHAR(20) NOT NULL,
  seat VARCHAR(10),
  meal VARCHAR(50),
  luggage VARCHAR(50),
  date DATE NOT NULL,
  flight_number VARCHAR(50) NOT NULL,
  airline VARCHAR(100) NOT NULL,
  source VARCHAR(100) NOT NULL,
  destination VARCHAR(100) NOT NULL,
  travel_class VARCHAR(50) NOT NULL
) ENGINE=InnoDB;

CREATE INDEX idx_user_email ON users (email);
CREATE INDEX idx_flight_number ON flights (flight_number);
CREATE INDEX idx_booking_user_id ON my_bookings (user_id);
CREATE INDEX idx_booking_flight_id ON my_bookings (flight_id);
CREATE INDEX idx_booking_status ON my_bookings (status);

```

```

1 • DROP DATABASE IF EXISTS aeroo_connect;
2 • CREATE DATABASE aeroo_connect;
3 • USE aeroo_connect;
4
5 • CREATE TABLE userrs (
6   id INT AUTO_INCREMENT PRIMARY KEY,
7   name VARCHAR(100) NOT NULL,
8   email VARCHAR(255) NOT NULL UNIQUE,
9   password VARCHAR(255) NOT NULL,
10  phone VARCHAR(20) NULL,
11  otp VARCHAR(10) NULL,
12  otp_expiry DATETIME NULL,
13  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
14 );
15 • ALTER TABLE userrs MODIFY phone VARCHAR(20) NOT NULL UNIQUE;
16

```

b. Authentication / Authorization

1. Authentication

Authentication is the process of verifying a user's identity — making sure the user is who they claim to be.

A user (passenger) or admin must log in with valid email/username and password.

- After successful login, the system generates a token (usually using JWT – JSON Web Token).
- The token is stored on the client side (like in localStorage) and sent with future requests.

Purpose:

- To ensure only registered users can access services like searching, booking, or viewing bookings.

2. Authorization

Authorization determines what actions or resources a user is permitted to access, after authentication is complete.

- A user can:
 - o Search and book flights
 - o Make payments
 - o View or cancel their own bookings
- An admin can:
 - o Add/edit/delete flights
 - o View all user bookings
 - o Manage system data

c. Data Store / Retrieval / Update

1. Data Storage (Store)

Saving user, flight, booking, and payment data permanently in a MySQL database.

Examples:

- When a user registers, their info is stored in the users table.
- When an admin adds a flight, it's stored in the flights table.
- When a passenger books a flight, booking info is stored in the bookings table.

Key Tables:

- users, flights, bookings, payments

2. Data Retrieval

Fetching data from the database for display or processing.

Examples:

- When a user logs in, their credentials are checked from the users table.
- When a user searches for flights, available flights are fetched from the flights table.
- When an admin views all bookings, records are fetched from bookings.

3. Data Update

Modifying existing records in the database.

Examples:

- User updates their profile details.
- Admin updates flight departure time.
- Payment status changes from Pending to Paid.

d. Data Validation

Data Validation is the process of ensuring that the data entered by users or systems is:

- Correct
- Complete
- Secure
- In the proper format

It helps prevent errors, data corruption, and security issues by verifying inputs before saving or processing them.

To ensure:

- Users enter valid email, phone number, and passwords
- Flights are added with proper times and cities
- Payments are recorded with correct amounts and booking IDs
- Hackers can't inject invalid or malicious input (like SQL injections)

e. Search Module

The Search Module allows users to find available flights based on specific criteria such as:

- Source & destination cities
- Travel date
- Airline
- Departure time
- Class (Economy/Business)
- Price range

Example :

1. User enters Delhi → Mumbai, 2025-06-21
2. Selects filter: Non-stop only, Before 12PM
3. Clicks “Search Flights”
4. Gets a filtered list of available flights

f. Named Procedures / Functions

Named procedures and functions are predefined blocks of code used to perform specific tasks like booking, calculating price, or validating input.

Stored Procedure (in MySQL):

- A group of SQL statements saved in the database.
- Example: Automatically insert a new booking.

g. Passing Parameters

Passing parameters means sending values (data) into a function, procedure, or API so it can perform a task based on that input.

- Allows dynamic operations (like booking a specific flight or user login).
- Makes code flexible and reusable.

Benefits:

- Task-specific output
- Reusability
- Easier maintenance

h. Backup & Recovery

i. Backup

Backup is the process of creating a copy of your database to prevent data loss in case of:

- Server failure
- Accidental deletion
- Cyber-attacks
- System crashes

It ensures that critical data like users, bookings, and payments can be restored when needed.

ii. Recovery

Recovery is the process of restoring data from a backup to bring the system back to its last working state.

Aero Connect System:

- Backup includes: users, flights, bookings, payments, admin tables.
- Daily/Weekly backups can be scheduled using tools like:
- mysqldump (for MySQL)
- Cron jobs or cloud-based backup tools
- Recovery involves importing the backup .sql file into the database.

i. Internal Documentation

Internal Documentation refers to comments, naming conventions, and structure written inside the code to help developers understand, maintain, and debug the system easily.

Purpose in Aero Connect:

- Helps team members understand the logic of flight search, booking, payment, etc.
- Makes the codebase maintainable and scalable
- Useful for future enhancements or handovers

Front-end Code

Client/src/components/HomePage.js

```
import React, { useState } from 'react';

import { Link, useNavigate } from 'react-router-dom';

import './HomePage.css';

import logo from '../assets/logo.png';

import { FaUserCircle } from 'react-icons/fa';

import { useUser } from './UserContext';

import VoiceInput from './VoiceInput';

const HomePage = () => {

  const { user } = useUser();

  const navigate = useNavigate();

  const [from, setFrom] = useState('');

  const [to, setTo] = useState('');

  const [departureDate, setDepartureDate] = useState('');

  const [passengers, setPassengers] = useState(1);

  const handleProfileClick = () => {

    if (user && user.profileComplete) {

      navigate('/profile');

    } else {

      navigate('/register');

    }

  }

}
```

```

};

const handleSearchFlights = () => {

  if (from.trim().toLowerCase() === to.trim().toLowerCase()) {

    alert('Departure and destination cities must be different.');
```

return;

```

  }

  const queryParams = new URLSearchParams();

  queryParams.append('from', from);

  queryParams.append('to', to);

  if (departureDate) {

    queryParams.append('date', departureDate);

  }

  queryParams.append('passengers', passengers);

  // Redirect to search-results instead of /flights

  navigate(`/search-results?${queryParams.toString()}`);

};

return (

  <div className="homepage-container">

    <header className="navbar">

      <div className="logo-title">

        <img src={logo} alt="Logo" className="logo" />

      </div>

      <nav className="nav-links">
```

```

<Link to="/">Home</Link>

<Link to="/register">Register</Link>

<Link to="/login">Login</Link>

<Link to="/flights">Flights</Link>

<div

  className="profile-icon-link"

  onClick={handleProfileClick}

  title="Profile"

>

  <FaUserCircle className="profile-icon" />

</div>

</nav>

</header>

<section className="hero-section">

  <div className="hero-text">

    <h1>Life Is Short And The World Is Wide!</h1>

    <p>

      To get the best of your adventure you just need to leave and go

      where you are waiting for

    </p>

  </div>

</section>

```

```

<section className="search-section">

  <div className="search-box">

    <div className="search-row">

      <VoiceInput

        label="Departure City"

        value={from}

        onChange={setFrom}

      />

      <VoiceInput

        label="Arrival City"

        value={to}

        onChange={setTo}

      />

    </div>

    <div className="search-row">

      <input

        type="date"

        value={departureDate}

        onChange={(e) => setDepartureDate(e.target.value)}

      />

      <input

        type="number"

        min="1"

```

```

        max="15"

        placeholder="Passengers"

        value={passengers}

        onChange={(e) => setPassengers(Number(e.target.value))}

    />

    <button className="search-btn" onClick={handleSearchFlights}>

        Search Flights ✈️

    </button>

</div>

</div>

</section>

<footer className="footer-links">

    <Link to="/help">Help</Link>

    <Link to="/about-us">About Us</Link>

</footer>

</div>

);

};

export default HomePage;

```

Client/public/confirmation.html

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8" />

  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>

  <title>Flight Booking Confirmation</title>

  <link rel="stylesheet" href="confirmation.css" />

  <style>

    .container {

      width: 80%;

      margin: 0 auto;

      font-family: Arial, sans-serif;

    }

    .booking-info {

      text-align: center;

    }

    .booking-info p {

      margin: 5px 0;

    }

    .booking-info strong {

      font-weight: bold;
```

```
}

.section {

    margin-top: 20px;

}

.section-title {

    font-weight: bold;

}

.cost-summary {

    margin-top: 20px;

}

.cost-item {

    margin: 5px 0;

    display: flex;

    justify-content: space-between;

    max-width: 300px;

    margin-left: auto;

    margin-right: auto;

}

.total {

    margin-top: 10px;

    font-size: 18px;

    display: flex;

    justify-content: space-between;
```

```
max-width: 300px;

margin-left: auto;

margin-right: auto;
}

.info-row {

display: flex;

justify-content: space-between;

align-items: center;

margin-bottom: 20px;
}

.info-row div {

width: 30%;

text-align: left;
}

.info-row-center {

width: 30%;

text-align: center;
}

.info-row div:first-child {

text-align: left;
}

.info-row div:last-child {

text-align: right;
```



```

}

.info-row .account-right {

    text-align: right;

}

.info-row .date-left {

    text-align: left;

    margin-left: -20px; /* Move the date more to the left */

}

.plane-image {

    text-align: center;

    margin-top: 15px;

    margin-bottom: 25px;

}

.plane-image img {

    width: 200px;

    height: auto;

}

/* Print and download button styles */

.buttons-container {

    margin-top: 30px;

    text-align: center;

}

.buttons-container button {

```

```

padding: 10px 20px;

font-size: 16px;

cursor: pointer;

margin: 0 10px 10px 10px;

background-color: #4CAF50;

color: white;

border: none;

border-radius: 5px;

transition: background-color 0.3s ease;
}

.buttons-container button:hover {

background-color: #45a049;

}

</style>

</head>

<body>

<div class="container">

<h1>Aero Connect Booking Confirmation</h1>

<div class="booking-section">

<h2>Flight Booking Confirmation</h2>

<div class="booking-info">

<div class="info-row">

<div class="date-left">

```

```

    <p><strong>Booking Date:</strong></p>

    <p id="booking-date"></p>

</div>

<div class="info-row-center">

    <p><strong>Passenger Name:</strong></p>

    <p id="passenger-name"></p>

</div>

<div class="account-right">

    <p><strong>Account Name:</strong></p>

    <p id="account-info"></p>

</div>

</div>

</div>

</div>

<div class="plane-image">

</div>

</div>

<h2>Flight Details</h2>

<div class="section">

    <div class="section-title">Route</div>

    <table style="width:100%; border-collapse: collapse;">

        <tr>

            <th style="border: 1px solid #ddd; padding: 8px;">Departure City</th>

```

```

<th style="border: 1px solid #ddd; padding: 8px;">Arrival City</th>

<th style="border: 1px solid #ddd; padding: 8px;">Airline</th>

<th style="border: 1px solid #ddd; padding: 8px;">Departure Time</th>

</tr>

<tr>

<td id="departure-city" style="border: 1px solid #ddd; padding: 8px;"></td>

<td id="arrival-city" style="border: 1px solid #ddd; padding: 8px;"></td>

<td id="airline" style="border: 1px solid #ddd; padding: 8px;"></td>

<td id="departure-time" style="border: 1px solid #ddd; padding: 8px;"></td>

</tr>

</table>

</div>

```

```

<div class="section">

<div class="section-title">Seating Information</div>

<table style="width:100%; border-collapse: collapse;">

<tr>

<th style="border: 1px solid #ddd; padding: 8px;">Travel Class</th>

<th style="border: 1px solid #ddd; padding: 8px;">Extra Luggage</th>

<th style="border: 1px solid #ddd; padding: 8px;">Seat Assignment</th>

</tr>

<tr>

<td id="travel-class" style="border: 1px solid #ddd; padding: 8px;"></td>

```

```

        <td id="extra-luggage" style="border: 1px solid #ddd; padding: 8px;"></td>

        <td id="seat-assignment" style="border: 1px solid #ddd; padding: 8px;"></td>

    </tr>

</table>

</div>

<div class="section">

    <div class="section-title">Cost Summary</div>

    <div class="cost-summary">

        <div class="cost-item"><span>Ticket Price</span><span id="ticket-price">$0</span></div>

        <div class="cost-item"><span>Airport Tax</span><span>$10</span></div>

        <div class="cost-item"><span>Additional Charges</span><span>$65</span></div>

        <div class="cost-item"><span>Fuel & Insurance</span><span>$25</span></div>

        <div class="cost-item"><span>Booking Fee</span><span>$5</span></div>

        <div class="total"><strong>Total Amount</strong><strong id="total-amount">$0</strong></div>

    </div>

</div>

<div class="notes">

    <h3>Key Notes</h3>

    <p>Please bring this confirmation, along with a government-issued photo ID, upon check-in.</p>

</div>

```

```

<!-- Buttons for Print, Download and Back -->

<div class="buttons-container">

  <button onclick="window.print()">Print Confirmation</button>

  <button id="download-btn">Download as PDF</button>

  <button onclick="window.location.href='/'">Back to Homepage</button>

</div>

</div>

<!-- jsPDF library -->

<script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf/2.5.1/jspdf.umd.min.js"></script>

<script>

window.onload = () => {

  const bookingData = JSON.parse(localStorage.getItem('bookingData'));

  if (bookingData) {

    document.getElementById('booking-date').textContent = new
Date().toLocaleDateString();

    document.getElementById('account-info').textContent = bookingData.email || 'N/A';

    document.getElementById('passenger-name').textContent = bookingData.name || 'N/A';

    document.getElementById('departure-city').textContent = bookingData.source || 'N/A';

    document.getElementById('arrival-city').textContent = bookingData.destination || 'N/A';

    document.getElementById('airline').textContent = bookingData.airline || 'N/A';

    document.getElementById('departure-time').textContent = new
Date(bookingData.departureDate).toLocaleString() || 'N/A';

    document.getElementById('travel-class').textContent = bookingData.travelClass ||
'Economy';

```

```

        document.getElementById('extra-luggage').textContent = bookingData.extraLuggage ||
'None';

        document.getElementById('seat-assignment').textContent =
bookingData.seatAssignment || 'Unassigned';

        const ticketPrice = parseFloat(bookingData.price) || 0;

        const airportTax = 10;

        const additionalCharges = 65;

        const fuelInsurance = 25;

        const bookingFee = 5;

        const totalAmount = ticketPrice + airportTax + additionalCharges + fuelInsurance +
bookingFee;

        document.getElementById('ticket-price').textContent = `${ticketPrice.toFixed(2)};

        document.getElementById('total-amount').textContent = `${totalAmount.toFixed(2)};

    } else {

        alert('No booking data found in localStorage.');
```

```
    }
```

```
};
```

```
// Download as PDF using jsPDF
```

```
document.getElementById('download-btn').onclick = function() {
```

```
    const { jsPDF } = window.jspdf;
```

```
    const doc = new jsPDF();
```

```
    doc.text('Flight Booking Confirmation', 20, 20);
```

```
    doc.text(Booking Date: ${document.getElementById('booking-date').textContent}, 20, 30);
```

```

    doc.text(Passenger Name: ${document.getElementById('passenger-name').textContent},
20, 40);

    doc.text(Account Name: ${document.getElementById('account-info').textContent}, 20, 50);

    doc.text(Departure City: ${document.getElementById('departure-city').textContent}, 20,
60);

    doc.text(Arrival City: ${document.getElementById('arrival-city').textContent}, 20, 70);

    doc.text(Airline: ${document.getElementById('airline').textContent}, 20, 80);

    doc.text(Departure Time: ${document.getElementById('departure-time').textContent}, 20,
90);

    doc.text(Travel Class: ${document.getElementById('travel-class').textContent}, 20, 100);

    doc.text(Extra Luggage: ${document.getElementById('extra-luggage').textContent}, 20,
110);

    doc.text(Seat Assignment: ${document.getElementById('seat-assignment').textContent},
20, 120);


    doc.text('Cost Summary', 20, 130);

    doc.text(Ticket Price: ${document.getElementById('ticket-price').textContent}, 20, 140);

    doc.text(Airport Tax: $10, 20, 150);

    doc.text(Additional Charges: $65, 20, 160);

    doc.text(Fuel & Insurance: $25, 20, 170);

    doc.text(Booking Fee: $5, 20, 180);

    doc.text(Total Amount: ${document.getElementById('total-amount').textContent}, 20,
190);


    doc.save('flight-booking-confirmation.pdf');

```

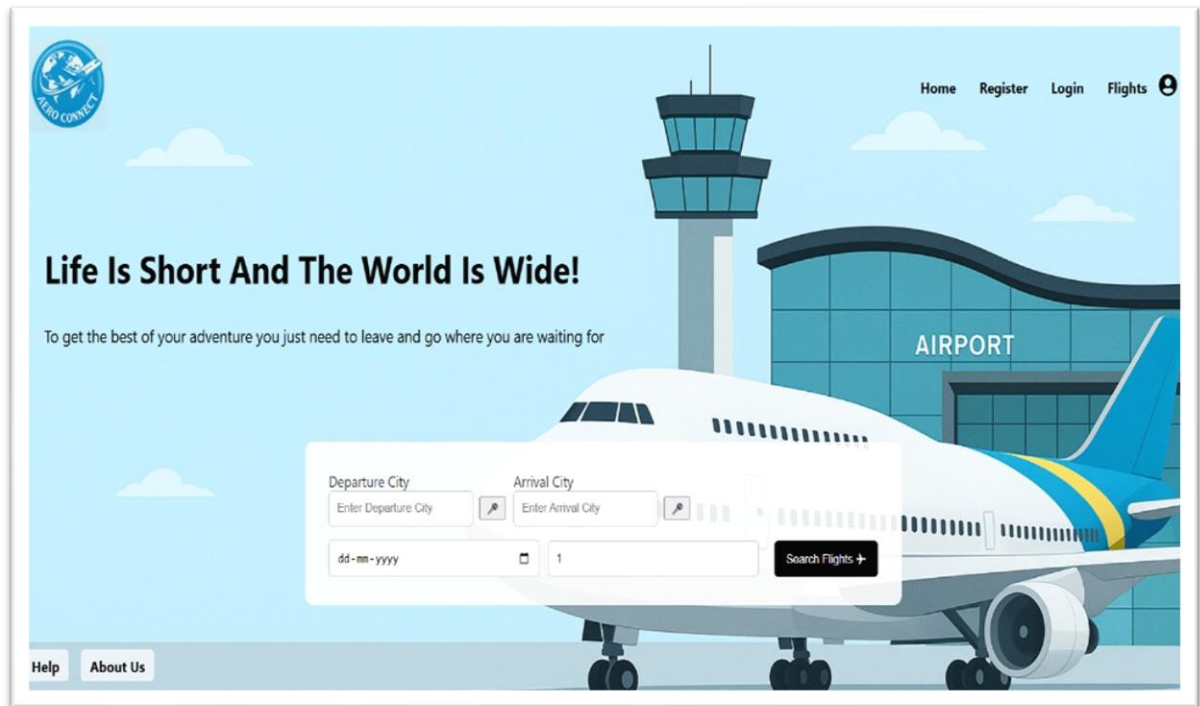


```
};  
  
</script>  
  
</body>  
  
</html>
```

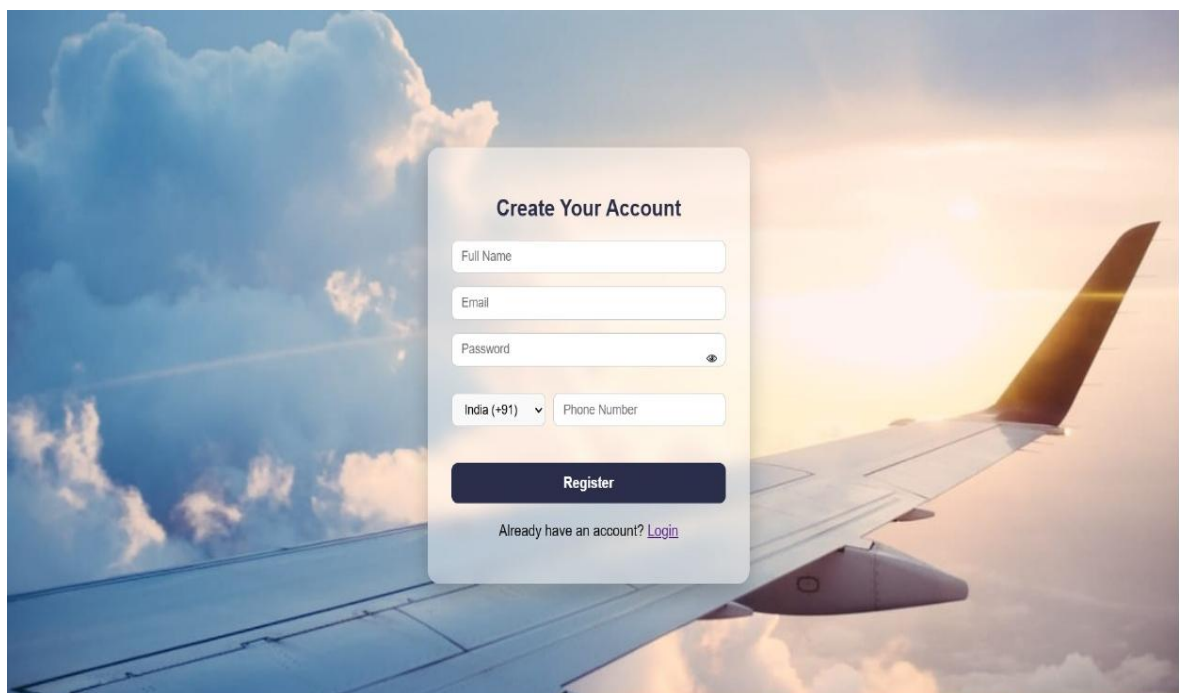
CHAPTER-7

User Interface (Screens and Reports)

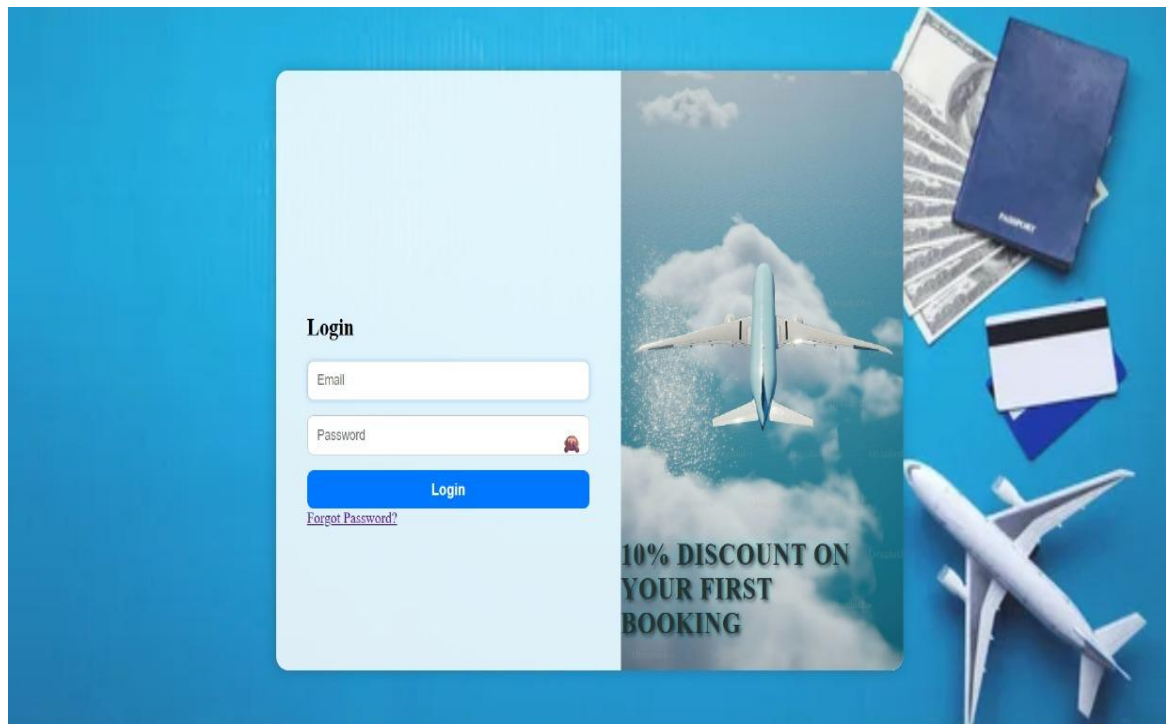
1. HOME PAGE



2. REGISTRATION PAGE



3.LOGIN PAGE



4. ADMIN DASHBOARD

Admin Dashboard - Flight Management

airline flight number departure city

arrival city dd-mm-yyyy --:-- dd-mm-yyyy --:--

price seats available

Add Flight

All Flights

Airline	Flight No.	From	To	Departure	Arrival	Price	Seats	Actions
Air India	A2101	Mumbai	Bangalore	5/20/2025, 11:28:00 AM	5/20/2025, 12:30:00 PM	\$4200.00	18	Edit Delete
Emirates	GS2727	Bangalore	Dubai	6/6/2025, 8:30:00 AM	6/6/2025, 9:45:00 AM	\$11500.00	25	Edit Delete
IndiGo	FS567	Jaipur	Delhi	5/21/2025, 8:00:00 PM	5/21/2025, 9:30:00 PM	\$6000.00	12	Edit Delete
AirIndia	AI576	Delhi	Mumbai	5/23/2025, 10:35:00 AM	5/23/2025, 11:32:00 AM	\$4500.00	16	Edit Delete

5. AVAILABLE FLIGHTS

Available Flights

Mumbai — Bangalore	Book Now
Departs: 5/20/2025, 11:28:00 AMArrives: 5/20/2025, 12:30:00 PM	
Bangalore — Dubai	Book Now
Departs: 6/6/2025, 8:30:00 AMArrives: 6/6/2025, 9:45:00 AM	
Jaipur — Delhi	Book Now
Departs: 5/21/2025, 8:00:00 PMArrives: 5/21/2025, 9:30:00 PM	
Delhi — Mumbai	Book Now
Departs: 5/23/2025, 10:35:00 AMArrives: 5/23/2025, 11:32:00 AM	
Shrinagar — Goa	Book Now
Departs: 5/30/2025, 4:20:00 PMArrives: 5/30/2025, 5:35:00 PM	

6. BOOKING PAGE

Book Your Flight

Flight Number: A2101
Airline: Air India
From: Mumbai
To: Bangalore
Departure: 5/20/2025, 11:28:00 AM
Arrival: 5/20/2025, 12:30:00 PM

Luggage (kg, up to 30):

Extra Luggage (kg, up to 3):

Travel Date:

Confirm Booking

7. CONFIRMATION PAGE

Aero Connect Booking Confirmation

Flight Booking Confirmation

Booking Date:
5/15/2025

Passenger Name:
Kasturi Patil

Account Name:
kvpati0210@gmail.com



Flight Details

Route			
Departure City	Arrival City	Airline	Departure Time
Mumbai	Bangalore	Air India	5/20/2025, 5:30:00 AM

Seating Information

Travel Class	Extra Luggage	Seat Assignment
Economy	None	Window

Cost Summary

Ticket Price	4200.00
Airport Tax	\$10
Additional Charges	\$65
Fuel & Insurance	\$25
Booking Fee	\$5
Total Amount	4305.00

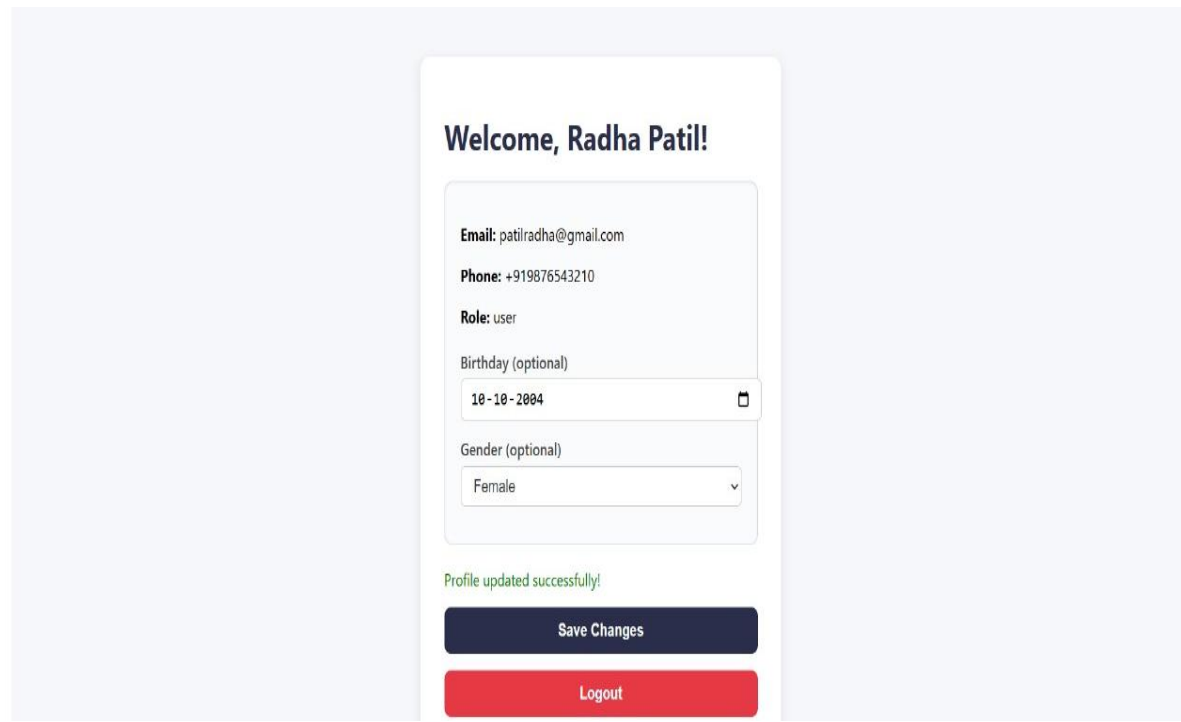
Key Notes

Please bring this confirmation, along with a government issued photo ID, upon check-in.

Print Confirmation

Download as PDF

8. PROFILE



A user profile update form for Radha Patil. The form is centered on a light gray background. It features a white card with a dark blue header "Welcome, Radha Patil!". Below the header, the user's details are listed: Email (patilradha@gmail.com), Phone (+919876543210), Role (user), Birthday (optional, 10-10-2004), and Gender (optional, Female). A green message "Profile updated successfully!" is displayed below the form. At the bottom, there are two buttons: "Save Changes" (dark blue) and "Logout" (red).

Welcome, Radha Patil!

Email: patilradha@gmail.com

Phone: +919876543210

Role: user

Birthday (optional)

10-10-2004

Gender (optional)

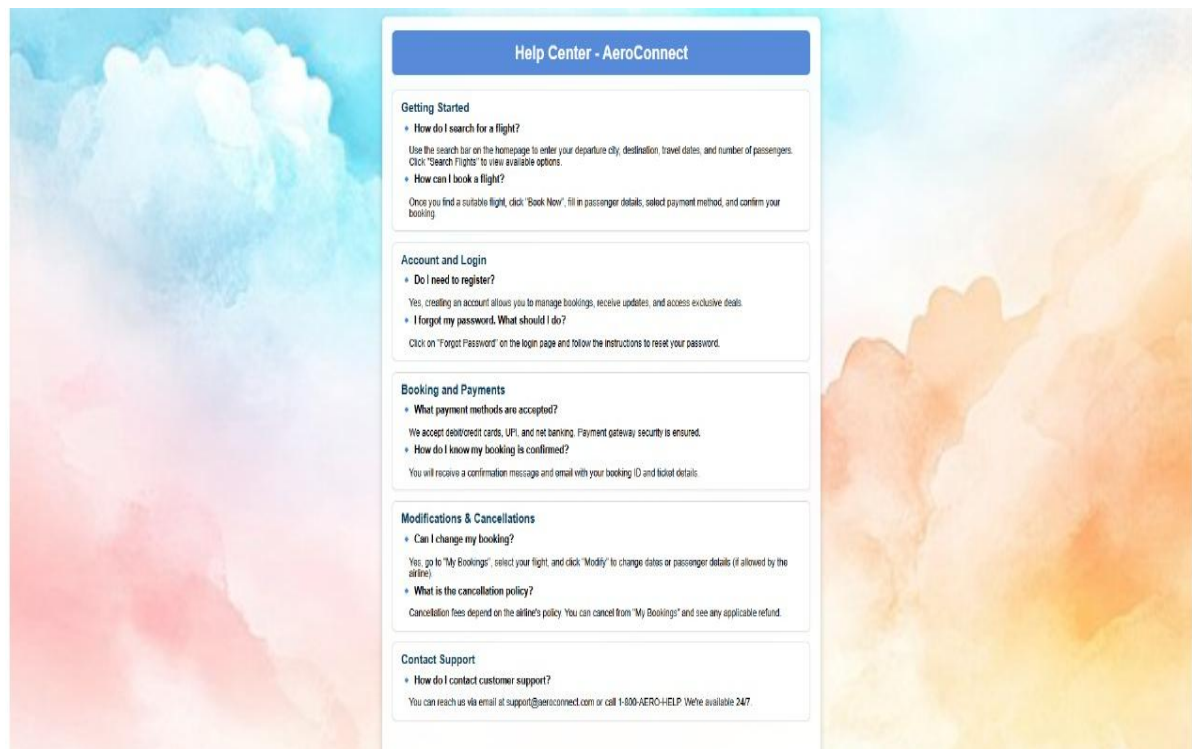
Female

Profile updated successfully!

Save Changes

Logout

9.HELP



A help center page for AeroConnect. The page has a blue header "Help Center - AeroConnect". The content is organized into five sections: Getting Started, Account and Login, Booking and Payments, Modifications & Cancellations, and Contact Support. Each section contains a list of questions and answers.

Help Center - AeroConnect

Getting Started

- **How do I search for a flight?**
Use the search bar on the homepage to enter your departure city, destination, travel dates, and number of passengers. Click "Search Flights" to view available options.
- **How can I book a flight?**
Once you find a suitable flight, click "Book Now", fill in passenger details, select payment method, and confirm your booking.

Account and Login

- **Do I need to register?**
Yes, creating an account allows you to manage bookings, receive updates, and access exclusive deals.
- **I forgot my password. What should I do?**
Click on "Forgot Password" on the login page and follow the instructions to reset your password.

Booking and Payments

- **What payment methods are accepted?**
We accept debit/credit cards, UPI, and net banking. Payment gateway security is ensured.
- **How do I know my booking is confirmed?**
You will receive a confirmation message and email with your booking ID and ticket details.

Modifications & Cancellations

- **Can I change my booking?**
Yes, go to "My Bookings", select your flight, and click "Modify" to change dates or passenger details (if allowed by the airline).
- **What is the cancellation policy?**
Cancellation fees depend on the airline's policy. You can cancel from "My Bookings" and see any applicable refund.

Contact Support

- **How do I contact customer support?**
You can reach us via email at support@aeroconnect.com or call 1-800-AERO-HELP. We're available 24/7.

10.ABOUT US

About Aero Connect

Established in 2025, Aero Connect is a cutting-edge flight booking platform committed to delivering a seamless and efficient travel experience.

Our Mission

Our mission is to empower travelers with a reliable, transparent, and user-friendly booking system, ensuring convenience and value at every step of the journey.

Why Choose Aero Connect?

- Secure and transparent booking process
- Exclusive deals and competitive pricing
- 24/7 dedicated customer support
- Intuitive and responsive user interface

Our Team

Comprised of experienced professionals in travel and technology, our team is dedicated to innovating and enhancing your travel experience.

Contact Us

For inquiries or feedback, please Contact Us @www.aeroconnect.com! We look forward to assisting you.

CHAPTER-8

TESTING

a. Introduction

Software testing is a vital phase in the software development lifecycle that ensures the quality, reliability, and correctness of a software application. It involves systematically evaluating the system to uncover any defects, missing functionalities, or gaps between the requirements and actual output. The goal of testing is to verify that the application performs as intended and meets user expectations.

For the **Aero Connect** software testing ensures that all modules — such as flight search, booking, user authentication, and payment — function accurately and securely. This step reduces the risk of failures, enhances user trust, and ensures a seamless experience for customers booking flights.

Testing is commonly divided into multiple levels, each serving a distinct purpose:

I. Unit Testing

Unit Testing is the initial level of testing that validates individual components or functions of the system in isolation. Each function or module is tested independently to ensure it performs correctly.

- **Tools Used:** Jest (JavaScript/React), Mocha (Node.js), JUnit (Java)
- **Focus:** Validating core logic of functions/modules such as fare calculation, user validation, and seat availability.
- **Example:** In Aero Connect, a unit test would check that the seat allocation function assigns available seats correctly and handles edge cases like overbooking or invalid seat numbers.

ii. Integration Testing

Integration Testing validates the interaction between modules to ensure they work together as expected. This stage detects issues in the interfaces and data exchange between components.

- **Tools Used:** Mocha, Jasmine, Postman (for APIs), Selenium
- **Focus:** Testing data flow between services — for example, booking a flight and linking it to the user profile and payment module.
- **Example:** In Aero Connect, integration tests would check that a flight booked through the frontend properly updates the backend database and triggers the email notification module.

iii. System Testing

System Testing is the final level where the entire integrated system is tested to verify that it meets the functional and non-functional requirements.

- **Tools Used:** Selenium (for UI testing), JMeter (for performance testing), Postman (for API testing)
- **Focus:** Complete end-to-end validation, including flight search, user login, booking, payment, and viewing booking history.
- **Example:** In Aero Connect, a system test would simulate a user booking a round-trip flight, making payment, and receiving a confirmation email — ensuring each step executes flawlessly.

b. Test Reports

Test reports provide a documented summary of the testing process and results. These reports help identify which modules are stable, which need further testing, and whether the software is ready for deployment.

Test Report Includes:

- **Test Summary:** Scope, objectives, and components tested.
- **Test Cases:** Each test case's input, expected output, actual result, and pass/fail status.

Conclusion

The **Aero Connect** is developed to revolutionize the process of booking and managing airline reservations by providing a centralized, efficient, and user-friendly digital platform. This system addresses the growing demand for streamlined air travel booking processes by automating key functions such as flight search, seat selection, reservation confirmation, payment processing, and ticket generation.

Throughout the development lifecycle of this project, we have systematically designed and implemented functionalities to ensure the platform meets both user expectations and industry standards. The documentation has thoroughly explored the system's scope, architecture, development strategies, and testing procedures, leading to a robust and scalable solution.

Aero Connect provides key features including real-time flight availability, dynamic fare calculations, secure user authentication, and integrated payment gateways. Each of these features has been built using modern web technologies, particularly the MERN stack (MongoDB, Express.js, React, and Node.js), ensuring flexibility, responsiveness, and maintainability.

The user interface is crafted to be intuitive and accessible across various devices, providing users — whether passengers, administrators, or agents — a seamless experience while interacting with the system. Moreover, the backend has been optimized to ensure high performance even under concurrent user load, which is a critical requirement for airline reservation systems.

The testing phase played a pivotal role in validating the Aero Connect system. Through unit, integration, and system testing, we ensured each component functions as expected individually and in tandem with others. Furthermore, security testing confirmed that

sensitive user data, including payment information and personal identification, is handled with robust encryption and authentication mechanisms.

In conclusion, Aero Connect stands out as a powerful, user-centric airline reservation system capable of enhancing the flight booking experience for customers while simplifying administrative operations for airline staff. The system's documentation ensures its maintainability and provides a blueprint for future upgrades and integrations, solidifying its long-term relevance and reliability in the aviation industry.

Limitations

While Aero Connect offers several advantages, there are some limitations to consider:

1. **Internet Dependency:** As a web-based platform, the system requires stable internet connectivity. In low-connectivity areas, the user experience may be impacted.
2. **Scalability under Extreme Load:** Although tested for concurrent users, massive spikes in traffic may require further infrastructure scaling and optimization.
3. **Integration with External APIs:** The system may face compatibility or maintenance issues when integrating with third-party services like global distribution systems (GDS), travel insurance APIs, or loyalty programs.
4. **User Onboarding and Training:** Airline staff and agents may need proper onboarding or training to use the admin features effectively.
5. **Limited Offline Functionality:** Currently, users cannot access their booking information offline, which could be enhanced in future updates.

Scope for Enhancement (Future Scope)

1. **Mobile Application:** Develop cross-platform Android/iOS apps for faster and more convenient access to bookings and flight updates.
2. **AI-based Price Prediction:** Implement machine learning models to suggest the best times to book based on historical fare trends.
3. **Integration with Loyalty Programs:** Allow users to collect and redeem points or miles via the platform.
4. **Multilingual Support:** Add language options for international users to enhance accessibility.
5. **Real-Time Flight Status Updates:** Integrate live updates for delays, gate changes, or cancellations.
6. **Voice Search and Chatbots:** Integrate virtual assistants for flight booking assistance and FAQs.
7. **Dynamic Seat Maps:** Visual seat selection with real-time updates based on user activity.
8. **Offline Booking Summary Access:** Allow users to view ticket information and QR boarding passes without internet access.

Abbreviations and Acronyms

- **API:** Application Programming Interface
- **JWT:** JSON Web Token
- **UI:** User Interface
- **UX:** User Experience
- **CI/CD:** Continuous Integration / Continuous Deployment
- **HTTPS:** HyperText Transfer Protocol Secure
- **GDS:** Global Distribution System
- **CRUD:** Create, Read, Update, Delete
- **OTP:** One-Time Password
- **SSL:** Secure Sockets Layer
- **REST:** Representational State Transfer
- **DBMS:** Database Management System
- **MERN:** MySQL, Express.js, React.js, Node.js
- **PDF:** Portable Document Format
- **2FA:** Two-Factor Authentication

Bibliography / References – Flight Reservation System

1. **Agile Software Development:** A comprehensive guide to agile methodologies and practices used in modern software projects like flight reservation systems.
 - Author: Robert C. Martin
 - Publisher: Prentice Hall
 - Year: 2003
2. **Clean Code:** A Handbook of Agile Software Craftsmanship: Principles and practices for writing clean, maintainable code used in scalable systems such as airline booking platforms.
 - Author: Robert C. Martin
 - Publisher: Prentice Hall
 - Year: 2008
3. **Designing Data-Intensive Applications:** A detailed exploration of the architecture and design of applications that handle large volumes of data, such as flight management and user reservations.
 - Author: Martin Kleppmann
 - Publisher: O'Reilly Media
 - Year: 2017
4. **Node.js Design Patterns:** Best practices and design patterns for building scalable and maintainable Node.js applications, essential for building backend services of flight booking systems.
 - Author: Mario Casciaro, Luciano Mammino
 - Publisher: Packt Publishing
 - Year: 2016
5. **MySQL Documentation:** Official documentation for MySQL, providing essential guidance on relational database operations for storing flights, bookings, users, and payment data.
 - URL: <https://dev.mysql.com/doc/>

6. **MDN Web Docs:** Comprehensive documentation for web technologies including HTML, CSS, and JavaScript, used in building responsive UIs for flight search and booking.
 - URL: <https://developer.mozilla.org/>
7. **Express.js Documentation:** Official documentation for Express.js, a fast and minimalist web framework for Node.js used in creating REST APIs for flight, user, and booking management.
 - URL: <https://expressjs.com/>
8. **Sequelize Documentation:** Official documentation for Sequelize, an ORM for Node.js used to map JavaScript objects to MySQL tables in the flight reservation system.
 - URL: <https://sequelize.org/docs/v6/>
9. **JWT Documentation:** Official documentation for JSON Web Tokens, used for secure authentication and authorization in user login and booking access in the airline system.
 - URL: <https://jwt.io/introduction>